

Problem Solving and Programming in C — Study Notes

Problem Solving and Programming in C — Study Notes

Course Code: B21CA02DC | **Semester:** I | **Programme:** BCA | **University:** Sreenarayanaguru Open University (SGOU)

Subject: Problem Solving and Programming in C | **Coverage:** Blocks 1–4, 16 Units (Theory Only)

Source: Self Learning Material (SLM) — SGOU BCA Semester I

About These Notes

Prepared by: Abdul Vahab A A

Website: <https://abdulvahabaa.in>

These notes were created for my personal study and revision. I am sharing them here because someone else might find them useful too.

Wishing you all the very best for your exams, lab work, and programming journey. Study with focus, practice programs regularly, and stay consistent — success will follow.

Duaon mein Yaad Rakhna.

Index

Block 1: Basic Programming Concepts — Problem solving, algorithms, C basics, variables, operators

Block 2: I/O, Control Structures, Arrays and Pointers — I/O functions, loops, arrays/strings, pointers

Block 3: Functions, Structures and Union — Functions, recursion, parameters, structures/union

Block 4: Storage Classes, Files and Preprocessors — Storage classes, file handling, CLI args, preprocessor

(Detailed subsection numbers match SGOU SLM — see each unit heading in the notes.)

Block 1: Basic Programming Concepts

Unit 1: Problem Solving and Algorithms

1.1.1 Problem-solving

Theory

Problem-solving is the process of identifying a problem, analysing its requirements, and finding a systematic solution that can be implemented on a computer. In programming, problem-solving is the foundation — before writing any code, the programmer must fully understand what is being asked, what data is involved, what output is expected, and what constraints exist (such as time, memory, or input limits).

Computers are fast and accurate but they cannot think independently. They follow instructions exactly as given. Therefore, a human programmer must break a complex real-world problem into smaller, logical steps that a computer can execute. Problem-solving in computing typically involves understanding the problem statement, identifying inputs and outputs, choosing an appropriate method, designing an algorithm, writing the program, and then testing and debugging the result.

Good problem-solving skills include logical thinking, attention to detail, patience during debugging, and the ability to test edge cases (such as zero values, negative numbers, or empty input). Many programming errors occur not because of syntax mistakes but because the problem was not understood correctly in the first place.

Important Points

- Problem-solving = analyse → design solution → implement → test → debug
- Computers execute instructions literally; they do not infer intent
- Clear understanding of inputs, outputs, and constraints is essential before coding
- Debugging is part of problem-solving, not a separate activity
- Real-world problems must be converted into computational steps

For Exam

Problem-solving is the systematic process of understanding a problem, analysing its requirements, and developing a step-by-step solution suitable for computer implementation. It involves identifying inputs and outputs, breaking the problem into smaller parts, designing an algorithm, writing the program in a language like C, and testing the result. Since computers cannot think on their own, the programmer must provide precise, unambiguous instructions. Effective problem-solving requires logical reasoning, careful analysis, and iterative testing until the correct output is obtained.

1.1.2 Approaches in problem-solving

Theory

There may be multiple solutions and techniques available to solve a problem using a computer. Mainly there are two popular approaches for problem-solving: the **top-down approach** and the **bottom-up approach**. Both help manage complexity in large programs, but they start from opposite directions.

Choosing the right approach depends on the problem. If you need a clear overall plan first, top-down is better. If you already have tested small modules (like ready-made functions), bottom-up is faster. In C programming, we often plan top-down and implement bottom-up by writing separate functions.

Important Points

- Two main approaches: **top-down** and **bottom-up**
- Top-down: whole problem → smaller parts
- Bottom-up: small modules → complete program
- Both reduce complexity of large programs
- C uses modular programming combining both ideas

For Exam

Two popular approaches in problem-solving are top-down and bottom-up. Top-down starts with the whole problem and breaks it into smaller sub-problems. Bottom-up builds small basic modules first and combines them into a complete solution. Both approaches help manage complexity in program design.

1.1.3 Top-Down Approach

Theory

In the **top-down approach**, we first understand the whole problem without going into all details. We formulate an overall design for the solution, and then move to the details as required.

Example from the SLM: paying a gardener daily wages using a gold bar cut into seven parts — first we analysed the full question, then broke it into step-by-step logic, and only then implemented the solution. That is top-down thinking.

In programming, top-down design means the main function or main module controls the overall flow, and sub-tasks are handled by subprograms (functions). This matches structured programming in C.

Important Points

- Understand **entire problem first**, then details
- Break main problem into **sub-problems** recursively

- Main module controls sub-modules
- Good for planning and documentation
- Used in structured C programming

For Exam

In the top-down approach, the programmer first understands the whole problem and creates an overall design. Then the problem is divided into smaller sub-problems until each part is simple enough to solve. It is like reading a full exam question before writing section-wise answers. Top-down design gives a clear hierarchical structure to the program.

1.1.4 Bottom-Up Approach

Theory

In the **bottom-up approach**, we find the smallest module, solve it first, and then integrate all such modules to get the whole solution.

Real-life examples: manufacturing a car (parts made separately, then assembled), building a house (foundation, walls, roof built as parts), or a college day event (separate committees for reception, cultural events, etc. — each committee works independently but together makes the event successful).

In C, bottom-up means writing small functions first (e.g., add, subtract, multiply, divide for a calculator) and then calling them from `main()` to build the full program.

Important Points

- Build **smallest modules first**
- Integrate modules to form complete solution
- Example: calculator — separate functions for +, -, ×, ÷
- Promotes **reuse** of tested code
- Useful when basic components already exist

For Exam

In the bottom-up approach, small modules are developed and tested first, then combined to form the complete solution. For example, in a calculator program, separate functions for addition, subtraction, multiplication, and division are written first and then integrated in the main program. It is like assembling car parts into a complete vehicle.

1.1.5 Steps involved in problem-solving

Theory

Systematic problem-solving in programming follows a sequence of well-defined steps. Although textbooks may phrase them slightly differently, the core process remains the same.

Step 1 — Analyse the problem: Read the problem carefully. Identify what is given (input), what must be produced (output), and any special conditions. Clarify doubts before proceeding.

Step 2 — Design the solution: Develop an algorithm or flowchart representing the logic. Decide which formulas, data structures, and control structures are needed. This step does not involve writing C syntax yet — it focuses on logic.

Step 3 — Code the solution: Translate the algorithm into a C program using correct syntax, proper variable names, and appropriate library functions. Follow structured programming principles.

Step 4 — Test and evaluate: Run the program with sample inputs including normal cases, boundary cases, and invalid cases. Compare output with expected results. If errors appear, debug by tracing variables and checking logic. Testing and debugging together form the evaluation phase.

These steps may repeat cyclically — debugging often sends the programmer back to the design or coding stage. Documentation (comments and clear naming) throughout makes each step easier.

Important Points

- **Step 1:** Analyse — understand input, output, constraints
- **Step 2:** Design — algorithm/flowchart (logic, not syntax)
- **Step 3:** Code — write C program from design
- **Step 4:** Test and evaluate — run, verify, debug
- Steps are iterative; debugging may require redesign
- Skipping analysis or design leads to more errors later

For Exam

The four main steps in problem-solving for programming are: (1) Analyse the problem — understand inputs, outputs, and conditions; (2) Design the solution — create an algorithm or flowchart; (3) Code the solution — write the program in C; (4) Test and evaluate — execute the program, compare results, and debug errors. These steps may be repeated until the program works correctly for all required test cases.

1.1.6 Algorithm

Theory

An **algorithm** is a finite sequence of well-defined, unambiguous instructions written in step-by-step form to solve a specific problem. The word comes from the name of the Persian mathematician Al-Khwarizmi. Algorithms exist independently of any programming language — they describe **what** to do, not **how** to write it in C.

Every algorithm has three basic parts: **Input** (data supplied to the algorithm), **Process** (computations and decisions performed on the input), and **Output** (the result produced). For example, an algorithm to find the area of a rectangle takes length and breadth as input, multiplies them in the process, and gives area as output.

Algorithms can be expressed in plain English, pseudocode, flowcharts, or program code. Before coding, writing the algorithm helps detect logical errors early. A good algorithm should be correct (produces right output), efficient (uses reasonable time and memory), and easy to understand.

Example — algorithm to find the largest of two numbers:

1. Start
2. Read two numbers A and B
3. If $A > B$, set $MAX = A$; otherwise set $MAX = B$
4. Print MAX
5. Stop

Important Points

- Algorithm = finite, step-by-step, unambiguous instructions
- Three parts: **Input**, **Process**, **Output**
- Independent of programming language
- Can be written in English, pseudocode, or flowchart
- Must terminate after finite number of steps
- Helps detect logic errors before coding

An algorithm is a sequence of activities to be processed for getting the desired output from a given input. It comprises a set of unambiguous rules and has a clear stopping point.

Characteristics of a good algorithm:

Finiteness: Must terminate after a limited number of steps.

Definiteness (Unambiguity): Each step has exactly one meaning.

Input: Zero or more input values.

Output: At least one output result.

Effectiveness: Each step is practically executable.

Correctness: Produces correct output for valid input.

Important Points

- Algorithm = finite, step-by-step, unambiguous instructions
- Three parts: Input, Process, Output
- Must be **finite** and **effective**
- Independent of programming language
- Notation in algorithms: + - * / and ← for assignment
- Helps detect logic errors before coding

For Exam

An algorithm is a finite set of clear, step-by-step instructions to solve a problem. It has input, process, and output. A good algorithm must be finite, definite, effective, and correct. For example, to find the largest of two numbers: read A and B, compare them, assign the larger to MAX, and print MAX.

1.1.7 Flowchart

Theory

A **flowchart** is a pictorial or graphical representation of an algorithm. It uses different symbols, shapes, and arrows to represent the process flow. Once a flowchart is made, it is easy to convert it into any programming language.

Flowcharts help in communicating logic clearly and in checking whether all cases are handled before coding.

Important Points

- Flowchart = graphical/pictorial form of algorithm
- Easier to understand than text for complex logic
- Can be converted directly into C code
- Used in program planning and documentation

For Exam

A flowchart is a pictorial representation of an algorithm using standard symbols connected by flow lines. It shows the step-by-step logic of a problem visually. Flowcharts make complicated problems easier to understand and help in converting logic into a program.

1.1.7.1 Symbols used in a flowchart

Theory

Each flowchart symbol has a specific meaning:

Symbol	Name	Function
Oval	Terminator	Starting or end of program
Parallelogram	Input/Output	Read data or display output
Rectangle	Process	Internal operation or calculation
Diamond	Decision	Yes/No or True/False question
Arrow	Flow line	Direction of flow
Circle	Connector	Connect distant parts without crossing lines
Rectangle (double)	Predefined process	Call a subroutine

Important Points

- **Rectangle** = process/operation (not same as parallelogram)
- **Parallelogram** = input/output only
- **Diamond** = decision with two exits
- **Oval** = start and stop
- Exam trap: rectangle and parallelogram are **not** interchangeable

For Exam

Standard flowchart symbols are: oval for start/stop (terminator), parallelogram for input/output, rectangle for processing, diamond for decision, and arrows for flow direction. The connector (circle) links distant parts of the chart. Rectangle is for operations; parallelogram is for input/output only.

1.1.7.2 Rules for flowcharting

Theory

Rules for drawing correct flowcharts (from SLM):

1. All boxes must be connected with arrows.
2. Every flowchart must start and end with a **terminator** (oval).
3. All symbols have only **one entry point** at the top (except decision).
4. Exit points are at the **bottom**, except decision symbol (exits on sides or bottom).
5. General flow is **top to bottom**. Upward flow allowed but not more than three symbols.
6. Decision symbol has **two exit points** (Yes/No or True/False).

Important Points

- One Start, one Stop — mandatory
- One entry per symbol (top)
- Decision = two exits
- Top-to-bottom flow preferred
- Sequential flow in algorithm = top to bottom in flowchart

For Exam

Rules for flowcharting: connect all symbols with arrows; start and end with terminator; each symbol has one entry at top; decision has two exits; flow is generally top to bottom. These rules ensure the flowchart correctly represents the algorithm logic.

1.1.8 Evolution of Programming languages

Theory

A **programming language** is a formal language used to write instructions (programs) that a computer can execute. Programming languages bridge the gap between human thinking and machine execution. They evolved because writing programs directly in binary (machine code) is extremely difficult, error-prone, and slow for humans.

Programming languages are classified into levels:

Low-level languages are close to machine hardware. They include machine language and assembly language. Programs written in these languages run fast but are hard to write and not portable across different computers.

High-level languages (such as C, Java, Python) use English-like words and mathematical notation. They are easier to learn, read, and maintain. A single high-level statement may correspond to many machine instructions. High-level programs require translation (compilation or interpretation) before execution.

C is considered a **middle-level language** because it combines high-level structured features with low-level memory access through pointers, giving both readability and control over hardware.

Important Points

- Programming language = medium to write computer programs
- **Low-level:** close to machine (machine lang, assembly)
- **High-level:** English-like, easier to use (C, Java, Python)

- C is **middle-level** — structured yet allows low-level control
- High-level programs need translators (compiler/interpreter)
- Higher level = more portable, generally slower development of raw speed

For Exam

Programming languages are formal languages for writing computer programs. They are classified as low-level (close to machine, e.g., machine and assembly language) and high-level (English-like, e.g., C, Java). Low-level languages are fast but difficult; high-level languages are easier but need translation. C is a middle-level language offering both structured programming and low-level memory control through pointers.

1.1.8.1 Machine languages

Theory

Machine language is the lowest-level programming language and the only language a computer's CPU understands directly. It consists entirely of **binary digits (0 and 1)** — each instruction is represented as a pattern of bits. Every processor family (such as Intel x86, ARM) has its own machine language instruction set.

Machine language instructions specify operations such as load data into a register, add two numbers, store result in memory, or jump to another instruction address. Because humans find binary extremely difficult to read and write, machine language programming is almost never done manually today.

Programs in machine language execute at maximum speed because no translation is needed. However, disadvantages include: very difficult to learn and debug, machine-dependent (not portable), tedious to write even for simple tasks, and prone to errors where a single wrong bit changes the entire instruction meaning.

Important Points

- Machine language = binary (0s and 1s) only
- Directly understood by CPU — no translation needed
- Machine-dependent (specific to processor)
- Fastest execution but hardest to write and maintain
- First generation language
- Almost never written by hand today

For Exam

Machine language is the first-generation programming language consisting entirely of binary digits (0 and 1). It is the only language directly understood by the computer

hardware. Programs in machine language execute very fast but are extremely difficult to write, read, and debug. Machine language is machine-specific and not portable to other processors.

1.1.8.2 Assembly language

Theory

Assembly language is a low-level language that uses **mnemonic codes** (short English-like abbreviations) instead of raw binary. For example, **ADD**, **MOV**, **SUB**, and **LOAD** represent machine operations in readable form. Assembly is the second-generation programming language.

Each assembly instruction corresponds closely to one machine instruction. Programmers still need to know the hardware architecture (registers, memory addresses). An **assembler** translates assembly code into machine language object code.

Advantages over machine language: easier to read and write, easier to debug, uses symbolic names for memory locations. Disadvantages: still machine-dependent, slower development than high-level languages, requires detailed hardware knowledge.

Assembly is used today in embedded systems, device drivers, operating system kernels, and performance-critical code sections where direct hardware control is essential.

Important Points

- Assembly = mnemonic instructions (ADD, MOV, SUB)
- Second-generation language
- One assembly instruction ≈ one machine instruction
- Translated by **assembler**
- Easier than machine language but still hardware-specific
- Used in embedded systems, OS kernels, drivers
- Not portable across different CPUs

For Exam

Assembly language is a low-level programming language that uses mnemonic codes like ADD and MOV instead of binary. It is the second-generation language and is translated into machine code by an assembler. Assembly is easier to read than machine language but is still machine-dependent and requires knowledge of hardware. It is used where direct control over hardware is needed.

1.1.8.3 High-level language

Theory

High-level languages (C, C++, Java, Python, etc.) are more like English. They are programmer-friendly — easy to write, maintain, and debug. High-level languages are **machine-independent** (portable).

A **translator** (compiler or interpreter) converts high-level programs into machine language before execution. C is a high-level structured language that also allows low-level control (middle-level).

Natural languages have ambiguity (e.g., word “bank” = financial institution or riverside). High-level programming languages have **strict grammar** to remove ambiguity.

Important Points

- English-like syntax, easier for humans
- Machine-independent (portable)
- Needs compiler or interpreter
- Examples: C, Java, Python, C++
- Strict grammar — no ambiguity
- C is structured and portable

For Exam

High-level languages use English-like words and are easy to learn and maintain. They are machine-independent and require a translator to convert to machine code. Examples include C, Java, and Python. Unlike natural language, programming languages have strict rules to avoid ambiguity.

1.1.9 Translators

Theory

High-level language programs must be translated into machine language before execution. There are two main approaches: **compilation** and **interpretation**. Assembly language uses an **assembler**.

Compiler: Reads the **entire source program** at once, checks syntax errors (with line numbers), and produces **object code**. C uses a compiler. Example: gcc. Faster execution; must recompile after changes.

Interpreter: Translates and executes **one line at a time**. Stops at errors line by line. Slower; no object file. C is **not** interpreted.

Assembler: Converts assembly mnemonics (ADD, MOV) into machine language.

Source code = human-readable program (.c). **Object code** = machine output after translation.

Important Points

Translator	Works on	Method	Used for
Compiler	High-level	Entire program at once	C, C++
Interpreter	High-level	Line by line	Python, BASIC
Assembler	Assembly	Mnemonics → binary	Assembly lang

- Source code → translator → object code → linker → executable
- False exam statement: "Compiler converts machine to high-level" — **False**
- MOV, ADD = assembly **mnemonics**

For Exam

Translators convert source programs into machine language. A compiler translates the entire program at once and produces object code — C uses this. An interpreter translates one line at a time. An assembler converts assembly language into machine code. Source code is the human-written program; object code is the machine-readable output.

Unit 2: Introduction to C Programming

1.2.1 Introduction to C

Theory

C is a general-purpose, procedural programming language known for its efficiency, flexibility, and close relationship with system software. It was designed for writing operating systems, compilers, and other system-level software, but is equally used for application development. C programs are structured as a collection of functions, with `main()` serving as the mandatory entry point.

A C program is written as a text file with extension `.c`. It must be compiled before execution. C is a **case-sensitive** language — `printf` and `Printf` are treated as different identifiers. Statements in C end with a semicolon (`;`), and blocks of code are enclosed in curly braces `{ }`.

C provides rich built-in operators, control structures, and library functions while giving the programmer direct access to memory through pointers. This combination makes C both powerful and potentially dangerous if used carelessly — which is why structured programming discipline is emphasised.

Important Points

- C = general-purpose, procedural, compiled language
- File extension: `.c`
- **Case-sensitive** language
- Every program needs `main()` function
- Statements end with **semicolon (;)**
- Blocks enclosed in `{ }`
- Used for system software and applications

For Exam

C is a general-purpose, procedural programming language used for system and application development. C programs are saved with `.c` extension, are case-sensitive, and must contain a `main()` function as the entry point. Each statement ends with a semicolon. C is compiled before execution and provides efficient access to hardware through pointers.

1.2.2 History and development of C

Theory

C was developed by **Dennis Ritchie** at Bell Telephone Laboratories (AT&T) in **1972**. It evolved from an earlier language called **B**, which itself was derived from BCPL. The primary motivation was to rewrite the Unix operating system in a higher-level language than assembly, while retaining efficiency.

Ken Thompson and Dennis Ritchie worked on Unix; C became the language of Unix and gained widespread adoption. In **1978**, Brian Kernighan and Dennis Ritchie published *The C Programming Language* (K&R C), which became the de facto standard. The **ANSI C** standard was established in **1989** (also known as C89 or C90), followed by C99 and C11 revisions.

C influenced many later languages including C++, Java, C#, and Python's syntax. Its principles of structured programming, functions, and pointers remain fundamental in computer science education.

Important Points

- Developed by **Dennis Ritchie**, Bell Labs, **1972**
- Evolved from language **B** (from BCPL)
- Created to rewrite **Unix** OS
- K&R book published **1978**

- **ANSI C** standard: 1989 (C89/C90)
- Influenced C++, Java, and other languages

For Exam

C was developed by Dennis Ritchie at Bell Labs in 1972 for writing the Unix operating system. It evolved from the B language. The classic reference is Kernighan and Ritchie's book (1978). ANSI standardized C in 1989. C became widely popular for system programming and influenced many modern languages.

1.2.3 Features of C language

Theory

C possesses several features that explain its enduring popularity:

Simple and efficient: C has a relatively small set of keywords and constructs, yet is powerful enough for complex programs.

Structured language: Code is organised into functions and blocks, supporting top-down design and modular programming.

Rich set of operators: Arithmetic, relational, logical, bitwise, and assignment operators allow compact expressions.

Pointers: C provides direct memory address manipulation, enabling dynamic memory, arrays, strings, and efficient parameter passing.

Portability: C programs can be compiled on different platforms with minimal changes (standard libraries and compiler differences notwithstanding).

Recursion: Functions can call themselves, enabling elegant solutions to mathematical and tree problems.

Extensibility: Programmers can create their own functions and libraries; `#include` links standard and custom headers.

Middle-level language: Combines high-level structured features with low-level memory control.

Fast execution: Compiled code runs close to hardware speed.

Important Points

- Simple, efficient, structured language
- Rich operators and built-in library functions
- **Pointers** for memory control
- **Portable** across platforms
- Supports **recursion** and **modular programming**

- **Middle-level** — between assembly and high-level
- Fast compiled execution
- Case-sensitive; 32 keywords

For Exam

The main features of C are: simplicity and efficiency, structured programming with functions, rich operators, pointer support for memory control, portability, recursion, extensibility through user-defined functions, and fast execution as a compiled middle-level language. C combines high-level readability with low-level hardware access.

1.2.4 Structure of a C program

Theory

A C program is organised into several sections, each serving a specific purpose:

Documentation section: Comments at the top describing the program, author, and date. Not executed.

Link section: `#include` preprocessor directives that link header files (e.g., `#include <stdio.h>` for input/output functions).

Definition section: `#define` directives for symbolic constants and macros.

Global declaration section: Variables and function prototypes declared outside all functions, visible to the entire program.

main() function: Mandatory entry point where execution begins. Contains local declarations and executable statements.

Subprogram section: User-defined functions (definitions) that perform specific tasks called from main or other functions.

A minimal valid program needs `#include` (if using library functions) and `main()`. Comments use `//` (single line) or `/* */` (multi-line).

Important Points

Section	Example	Purpose
Documentation	<code>/* Author: ... */</code>	Description
Link	<code>#include <stdio.h></code>	Header files
Definition	<code>#define PI 3.14</code>	Constants/macros
Global	<code>int x;</code>	Global variables
main()	<code>int main(){...}</code>	Entry point
Functions	<code>int add(...){...}</code>	User-defined functions

- Comments: `//` or `/* */`
- `#include`, `#define` are preprocessor directives
- Exactly one `main()` per program

For Exam

A C program consists of: documentation (comments), link section (`#include`), definition section (`#define`), global declarations, the `main()` function (mandatory entry point), and user-defined functions. Preprocessor directives start with `#`. Comments are ignored by the compiler. Execution always begins at `main()`.

```
#include<stdio.h>
/* Documentation: Hello World program */
#define MSG"Hello"
int g = 10;           /* global declaration */
int main() {
    printf("%s\n", MSG);
    return 0;
}
```

1.2.5 Compilation process

Theory

The process of converting C source code into an executable involves several stages:

- 1. Creating source code:** Write and save the program as a `.c` file using an editor.
- 2. Preprocessing:** The preprocessor handles directives (`#include`, `#define`, `#ifdef`). Header files are inserted, macros expanded. Output is `.i` file (preprocessed source).
- 3. Compilation:** The compiler translates preprocessed code into **assembly language** (`.s` file).
- 4. Assembly:** The assembler converts assembly code into **object code** (`.o` or `.obj` file) — machine language modules.
- 5. Linking:** The linker combines object code with library functions (e.g., `printf` from `stdio` library) to produce the final **executable** (`.exe` on Windows, `a.out` on Unix/Linux).
- 6. Execution:** The loader loads the executable into memory and execution begins at `main()`.

Command example: `gcc program.c -o program` then `./program`. Errors can occur at compile time (syntax) or run time (logic, division by zero).

Important Points

- Pipeline: **Source** → **Preprocess** → **Compile** → **Assemble** → **Link** → **Execute**
- Preprocessor: handles # directives
- Compiler: source to assembly
- Assembler: assembly to object code
- Linker: object code + libraries → executable
- `gcc file.c -o output` — compile and link
- Syntax errors caught at compile time; logic errors at run time

For Exam

The C compilation process involves: (1) writing source code (.c), (2) preprocessing (#include, #define), (3) compilation to assembly, (4) assembly to object code, (5) linking with libraries to create executable, and (6) execution starting at main(). The command `gcc program.c -o program` compiles and links a C program.

Unit 3: Variables and Data Types

1.3.1 Character set

Theory

The **character set** of C is the collection of characters that the language recognises. C uses the **ASCII (American Standard Code for Information Interchange)** character set. Every character is stored as a number (ASCII code) internally — for example, 'A' is 65 and '0' is 48.

The C character set includes:

- **Letters:** A–Z, a–z (52 letters)
- **Digits:** 0–9 (used in numbers and identifiers)
- **White space:** blank space, tab (`\t`), newline (`\n`), form feed, carriage return
- **Special symbols:** + - * / = % & | ^ ~ ! < > () [] { } ; : , . # " ' ?

Only characters from this set may appear in C source files (except in string literals and comments where extended characters may appear depending on compiler).

Understanding the character set is the first step toward understanding **tokens** — the basic building blocks of C programs.

Important Points

- C uses **ASCII** character set
- Categories: letters, digits, whitespace, special symbols
- Each character stored as numeric ASCII code

- 'A' = 65, 'a' = 97, '0' = 48 (not zero)
- Only valid characters allowed in source code
- Foundation for tokens and identifiers

For Exam

The character set of C includes uppercase and lowercase letters, digits 0-9, whitespace characters (space, tab, newline), and special symbols such as +, -, *, /, and brackets. C uses the ASCII character set where each character has a numeric code. This character set forms the basis for all tokens in a C program.

1.3.2 Tokens

Theory

Tokens are the smallest individual units of meaning in a C program. The compiler groups characters into tokens during lexical analysis (first phase of compilation). C has six types of tokens:

1. Keywords — Reserved words with special meaning (e.g., `int`, `if`, `return`). Cannot be used as identifiers.

2. Identifiers — Names given by programmer to variables, functions, etc.

3. Constants — Fixed values such as `100`, `3.14`, `'A'`, `"hello"`.

4. Strings — Sequence of characters in double quotes.

5. Operators — Symbols performing operations: `+`, `-`, `*`, `/`, `=`, `==`, etc.

6. Special symbols — Punctuation: `(){}[]; , #`

The compiler ignores whitespace (except within strings and to separate tokens). For example, `int sum=10;` contains tokens: `int`, `sum`, `=`, `10`, `;`.

Important Points

- Token = smallest meaningful unit in C
- Six types: keywords, identifiers, constants, strings, operators, special symbols
- Compiler separates tokens during lexical analysis
- Whitespace separates tokens (ignored otherwise)
- `int x = 5;` → tokens: `int`, `x`, `=`, `5`, `;`

For Exam

Tokens are the smallest individual units in a C program. There are six types: keywords, identifiers, constants, strings, operators, and special symbols. The

compiler groups characters into tokens during lexical analysis. Whitespace is used to separate tokens but is otherwise ignored.

1.3.3 Keywords

Theory

Keywords (also called reserved words) are predefined tokens that have special meaning to the C compiler. They cannot be used as identifiers (variable or function names). C has **32 keywords** — all written in **lowercase**.

Common keywords include:

- Data types: `int`, `char`, `float`, `double`, `void`, `short`, `long`, `signed`, `unsigned`
- Control flow: `if`, `else`, `switch`, `case`, `default`, `for`, `while`, `do`, `break`, `continue`, `goto`
- Functions: `return`
- Storage class: `auto`, `extern`, `static`, `register`
- Others: `const`, `volatile`, `enum`, `struct`, `union`, `typedef`, `sizeof`

Using a keyword as a variable name (e.g., `int float = 5;`) causes a compilation error. Keywords are the vocabulary of the language — they define what operations and declarations are possible.

Important Points

- **32 keywords** in ANSI C — all lowercase
- Cannot be used as identifiers
- Examples: `int`, `char`, `if`, `else`, `for`, `while`, `return`, `void`
- Using keyword as name → **compile error**
- Include data type, control, storage class keywords

For Exam

Keywords are reserved words in C with special meaning to the compiler. C has 32 keywords, all in lowercase, such as `int`, `char`, `if`, `else`, `for`, `while`, `return`, and `void`. Keywords cannot be used as variable or function names. Using a keyword as an identifier causes a compilation error.

1.3.4 Identifiers

Theory

An **identifier** is a name given by the programmer to variables, functions, arrays, structures, and other user-defined elements. Identifiers allow meaningful names instead of memory addresses.

Rules for valid identifiers in C:

1. Must start with a **letter (A–Z, a–z)** or ****underscore (_)**** — not a digit.
2. Subsequent characters can be letters, digits, or underscores.
3. **No spaces** or special characters (except underscore).
4. **Cannot be a keyword.**
5. C is **case-sensitive** — `Total` and `total` are different identifiers.
6. Should be meaningful (e.g., `studentAge` rather than `x`).

There is no limit on length (compiler-dependent; first 31 characters significant in ANSI C). Valid: `sum`, `_count`, `avgMarks2`. Invalid: `2value` (starts with digit), `my-var` (hyphen), `int` (keyword).

Important Points

- Names for variables, functions, etc.
- Must start with letter or **underscore**
- Can contain letters, digits, underscore only
- **No spaces**, no special chars (except _)
- **Case-sensitive**; not a keyword
- Use meaningful names for readability

For Exam

Identifiers are user-defined names for variables, functions, and other program elements. Valid identifiers must start with a letter or underscore, followed by letters, digits, or underscores. They cannot be keywords, contain spaces, or start with a digit. C is case-sensitive, so Age and age are different identifiers.

1.3.5 Constants

Theory

A **constant** is a fixed value that does not change during program execution. C supports several types of constants:

Integer constants: Whole numbers. Forms: decimal (15), octal (017 — prefix 0), hexadecimal (0xF — prefix 0x). May have suffix u (unsigned) or l (long).

Real (floating-point) constants: Numbers with decimal point (3.14, 0.5) or in exponential form ($2.5e3 = 2500.0$).

Character constants: Single character in single quotes: 'A', '5', ". Stored as integer ASCII value.

String constants: Sequence of characters in double quotes: "Hello". Stored as array ending with null character '\0'.

Constants can be declared using `#define` (preprocessor) or `const` keyword:

`#define PI 3.14159` or `const float PI = 3.14159;`

Important Points

Type	Example	Notes
Integer decimal	100, -25	Base 10
Integer octal	017	Prefix 0
Integer hex	0x1F	Prefix 0x
Float	3.14, 2.5e-3	Decimal or exponential
Character	'A', '	Single quotes
String	"Hello"	Double quotes, ends with \0

- `#define N 100` — symbolic constant
- `const int x = 10;` — constant variable
- Character '5' ≠ integer 5

For Exam

Constants are fixed values that do not change during execution. Types include integer constants (decimal, octal with prefix 0, hex with prefix 0x), real/floating constants (3.14 or 2.5e3), character constants in single quotes ('A'), and string constants in double quotes ("Hello"). Constants can be defined using `#define` or the `const` keyword.

1.3.6 Data types and type modifiers

Theory

C requires every variable to be declared with a **data type** that specifies what kind of value it can store and how much memory it occupies.

Basic data types:

- `int` — integer numbers (typically 2 or 4 bytes)
- `char` — single character (1 byte)
- `float` — single-precision floating point (4 bytes)
- `double` — double-precision floating point (8 bytes)
- `void` — no value / no type (used with functions returning nothing)

Type modifiers alter the basic types:

- `short` — smaller int range
- `long` — larger int or double range
- `signed` — allows negative and positive (default for int)
- `unsigned` — only non-negative values (doubles positive range)

Examples: `unsigned int age;`, `long double precise;`, `short int count;`

The `sizeof()` operator returns the size in bytes of a type or variable. Exact sizes are compiler and platform dependent, but relative relationships (double > float, etc.) hold.

Important Points

Type	Size (typical)	Range/Purpose
char	1 byte	Single character
int	2 or 4 bytes	Integer
float	4 bytes	6-7 decimal digits
double	8 bytes	15 decimal digits
void	—	No type/value

- Modifiers: short, long, signed, unsigned
- `sizeof(type)` gives bytes occupied
- Uninitialized auto variables contain **garbage values**
- Always declare type before use in C

For Exam

Data types define the kind of values variables can hold. Basic types are int (integer), char (character), float and double (floating point), and void (no type). Modifiers include short, long, signed, and unsigned. For example, unsigned int stores only non-negative integers. The sizeof operator returns the memory size of a type in bytes.

Unit 4: Operators and Expressions

1.4.1 Introduction to operators

Theory

An **operator** is a symbol that tells the compiler to perform a specific operation on one or more **operands** (data values). For example, in `a + b`, `+` is the operator and `a`, `b` are operands.

C provides a rich set of operators classified as: arithmetic, relational, logical, assignment, increment/decrement, conditional (ternary), bitwise, and special operators. Operators make expressions — combinations that produce a value.

C operators follow **precedence** (priority order) and **associativity** (direction of evaluation when precedence is equal). Misunderstanding precedence causes bugs — use parentheses `()` to make order explicit when in doubt.

Operands can be variables, constants, or results of other expressions. The result of an operation can be assigned, compared, or used in further computations.

Important Points

- Operator performs operation on operand(s)
- C has arithmetic, relational, logical, assignment, bitwise, special operators
- **Precedence** determines evaluation order
- **Associativity**: left-to-right or right-to-left
- Use **()** to override default precedence
- Operands: variables, constants, expressions

For Exam

An operator is a symbol that performs an operation on operands. C provides arithmetic, relational, logical, assignment, increment/decrement, conditional, bitwise, and special operators. Operators follow precedence and associativity rules that determine evaluation order. Parentheses can be used to change the default order.

1.4.2 Arithmetic operators

Theory

C provides five arithmetic operators for numeric computations:

Operator	Name	Example	Result
+	Addition	5 + 3	8
-	Subtraction	5 - 3	2
*	Multiplication	5 * 3	15
/	Division	5 / 3	1 (integer division)
%	Modulus	5 % 3	2 (remainder)

Integer division: When both operands are integers, `/` truncates the decimal part. `9/2 = 4`, not 4.5. To get floating result, at least one operand must be float/double: `9.0/2 = 4.5`.

Modulus (`%`): Returns remainder after integer division. Works only with integers. `10 % 3 = 1`. Useful for checking even/odd (`n % 2`), cycles, and digit extraction.

Division by zero is undefined and causes runtime error. Arithmetic operators follow standard mathematical precedence: `*` / `%` before `+` -.

Important Points

- Five operators: `+`, `-`, `*`, `/`, `%`
- **Integer division** truncates: `9/2 = 4`
- Use `9.0/2` or `(float)9/2` for decimal result

- **Modulus %** — remainder; integers only
- Division by zero → runtime error
- - / % have higher precedence than + -

For Exam

C has five arithmetic operators: + (add), - (subtract), * (multiply), / (divide), and % (modulus/remainder). When both operands of / are integers, the decimal part is truncated (9/2=4). Modulus % gives the remainder and works only with integers. Division by zero causes a runtime error.

1.4.3 Relational operators

Theory

Relational operators compare two values and return **1 (true)** if the relation holds, or **0 (false)** if it does not. They are used in decision-making (`if` , `while`) and loops.

Operator	Meaning	Example	True when
<code>==</code>	Equal to	<code>a == b</code>	a equals b
<code>!=</code>	Not equal	<code>a != b</code>	a differs from b
<code>></code>	Greater than	<code>a > b</code>	a is larger
<code><</code>	Less than	<code>a < b</code>	a is smaller
<code>>=</code>	Greater or equal	<code>a >= b</code>	a at least b
<code><=</code>	Less or equal	<code>a <= b</code>	a at most b

Common error: using `=` (assignment) instead of `==` (comparison). `if (x = 5)` assigns 5 to x; `if (x == 5)` compares x with 5.

Relational operators have lower precedence than arithmetic operators but higher than logical operators. Result is always integer 0 or 1, never a boolean type (C has no native bool in older standards).

Important Points

- Compare two values; return 1 (true) or 0 (false)
- Operators: `==`, `!=`, `>`, `<`, `>=`, `<=`
- `==` compares; `=` assigns — common mistake
- Used in `if`, `while`, `for` conditions
- Result is int: 1 or 0
- Lower precedence than arithmetic

For Exam

Relational operators compare two values and return 1 if true, 0 if false. They are == (equal), != (not equal), > (greater), < (less), >= (greater or equal), and <= (less or equal). A common error is using = instead of == in conditions. They are used in if statements and loops.

1.4.4 Logical operators

Theory

Logical operators combine or modify conditions that evaluate to true (non-zero) or false (zero):

Operator	Name	Meaning	Example
&&	Logical AND	True if BOTH true	(a>0 && b>0)
	Logical OR	True if EITHER true	(a==0 b==0)
!	Logical NOT	Reverses condition	!(a > b)

Short-circuit evaluation: In `a && b`, if `a` is false, `b` is not evaluated. In `a || b`, if `a` is true, `b` is not evaluated. This can prevent errors (e.g., `ptr != NULL && *ptr > 0`).

In C, **0 is false; any non-zero value is true**. Logical expressions return 0 or 1. Logical operators have lower precedence than relational operators.

Important Points

- && (AND), || (OR), ! (NOT)
- **0 = false**, any non-zero = true
- AND: both must be true
- OR: at least one true
- NOT: reverses truth value
- Short-circuit: second operand may be skipped
- Lower precedence than relational

For Exam

Logical operators combine conditions: && (AND — both true), || (OR — either true), and ! (NOT — reverses). In C, zero is false and any non-zero is true. Logical operators use short-circuit evaluation — the second operand is evaluated only if needed. They return 0 or 1.

1.4.5 Assignment operators

Theory

The **assignment operator** `=` stores the value of the right-hand expression into the left-hand variable. C also provides **compound assignment operators** that combine an operation with assignment:

Operator	Equivalent	Example
<code>=</code>	simple assign	<code>a = 5</code>
<code>+=</code>	<code>a = a + b</code>	<code>a += 3</code>
<code>-=</code>	<code>a = a - b</code>	<code>a -= 2</code>
<code>*=</code>	<code>a = a * b</code>	<code>a *= 4</code>
<code>/=</code>	<code>a = a / b</code>	<code>a /= 2</code>
<code>%=</code>	<code>a = a % b</code>	<code>a %= 3</code>

Compound operators are shorthand and can produce slightly more efficient code. Assignment is **right-associative**: `a = b = c = 5` assigns 5 to c, then b, then a.

The left operand of assignment must be a modifiable lvalue (typically a variable). Constants and expressions like `(a+b) = c` are illegal.

Important Points

- `=` assigns value to variable
- Compound: `+=`, `-=`, `*=`, `/=`, `%=`
- `a += 3` same as `a = a + 3`
- Assignment is **right-associative**
- Left side must be variable (lvalue)
- Lower precedence than arithmetic and relational

For Exam

The assignment operator `=` stores a value in a variable. Compound assignment operators combine operation and assignment: `+=`, `-=`, `*=`, `/=`, and `%=` (e.g., `a += 3` means `a = a + 3`). Assignment is right-associative. The left operand must be a modifiable variable.

1.4.6 Increment and decrement operators

Theory

C provides unary **increment** (`++`) and **decrement** (`--`) operators to add or subtract 1 from a variable. They come in two forms:

Prefix (++a , --a): Increment/decrement **before** the value is used in the expression. If `a = 5`, then `b = ++a` gives `a = 6`, `b = 6`.

Postfix (a++ , a--): Use the **current value first**, then increment/decrement. If `a = 5`, then `b = a++` gives `b = 5`, `a = 6`.

Both forms change the variable by 1. They are commonly used in loop counters: `for (i = 0; i < 10; i++)`.

Applying `++` / `--` more than once to the same variable in one expression without sequence points leads to **undefined behaviour** — avoid such code.

Important Points

- `++` adds 1; `--` subtracts 1
- **Prefix** (`++a`): update first, then use value
- **Postfix** (`a++`): use value first, then update
- Example: `a=5; b=a++` → `b=5, a=6`
- Example: `a=5; b=++a` → `b=6, a=6`
- High precedence (like unary operators)
- Common in for loop third expression

For Exam

Increment (`++`) and decrement (`--`) operators add or subtract 1 from a variable. Prefix (`++a`) increments before use; postfix (`a++`) uses the current value then increments. For `a=5`: `b=a++` gives `b=5, a=6`; `b=++a` gives `b=6, a=6`. They are frequently used as loop counters.

1.4.7 Conditional (ternary) operator

Theory

The **conditional operator** `?:` is C's only ternary operator (three operands). Syntax: `condition ? expression_if_true : expression_if_false`

If the condition is true (non-zero), the first expression is evaluated; otherwise the second. Example: `max = (a > b) ? a : b;` assigns the larger of a and b to max.

It replaces simple if-else assignments in one line. Nested ternary operators are possible but reduce readability. The conditional operator has lower precedence than arithmetic and relational operators but higher than assignment.

Example: `grade = (marks >= 40) ? 'P' : 'F';` assigns 'P' for pass, 'F' for fail.

Important Points

- Syntax: `condition ? value_if_true : value_if_false`

- Only ternary operator in C
- Shorthand for simple if-else assignment
- Example: `max = (a > b) ? a : b;`
- Precedence: lower than arithmetic, higher than assignment
- Use for simple choices; if-else for complex logic

For Exam

The conditional (ternary) operator `?:` evaluates a condition and returns one of two expressions. Syntax: `condition ? expr1 : expr2`. If condition is true, `expr1` is used; otherwise `expr2`. Example: `max = (a > b) ? a : b` finds the maximum of two numbers in one statement.

1.4.8 Bitwise operators

Theory

Bitwise operators work on individual **bits** of integer operands. They cannot be used on float or double. Useful for low-level programming, flags, and embedded systems.

Operator	Name	Operation
<code>&</code>	AND	1 if both bits 1
<code> </code>	OR	1 if either bit 1
<code>^</code>	XOR	1 if bits differ
<code>~</code>	NOT	Flip all bits (unary)
<code><<</code>	Left shift	Shift bits left
<code>>></code>	Right shift	Shift bits right

Example: `5 & 3` → `0101 & 0011 = 0001 = 1`. Left shift `x << 1` doubles `x` (for positive ints). Right shift `x >> 1` halves `x`.

Bitwise operators have lower precedence than arithmetic but higher than logical. They are essential for masking, setting/clearing flag bits, and efficient multiply/divide by powers of 2.

Important Points

- Operate on **bits** of integers only
- `&`, `|`, `^` — binary; `~` — unary NOT
- `<<` left shift, `>>` right shift
- `x << 1` ≈ multiply by 2; `x >> 1` ≈ divide by 2
- Not for float/double

- Used in flags, embedded systems, masking

For Exam

Bitwise operators work on individual bits of integer operands: & (AND), | (OR), ^ (XOR), ~ (NOT), << (left shift), >> (right shift). They cannot be used on floating-point types. Left shift multiplies by powers of 2; right shift divides. They are used for flag manipulation and low-level programming.

1.4.9 Special operators

Theory

C includes several special operators beyond the common categories:

sizeof operator: Returns the size in bytes of a type or expression. `sizeof(int)` typically returns 4. Useful for portable memory calculations and dynamic allocation.

Address-of operator (&): Returns the memory address of a variable. `&x` gives where x is stored. Used with pointers and scanf.

Indirection/dereference operator (*): Accesses the value at an address stored in a pointer. If `p` points to x, `*p` is the value of x.

Comma operator (,): Evaluates expressions left to right; result is value of last expression. Used in for loops: `for (i=0, j=10; i<j; i++, j--)`.

Member operators (. and ->): Access structure/union members — covered in Block 3.

These operators are fundamental to pointers, structures, and memory management in C.

Important Points

- **sizeof** — bytes occupied by type/variable
- **&** — address-of (returns pointer)
- ******** — dereference (value at address)
- **,** — comma operator (left to right, last value)
- **.** and **->** — structure member access
- **sizeof** and **&** used before pointers chapter

For Exam

Special operators in C include: `sizeof` (returns size in bytes), `&` (address-of), `*` (dereference/value at address), comma operator (evaluates left to right), and member access operators (`.` and `->`) for structures. `sizeof` is used for memory calculations; `&` and `*` are essential for pointers.

1.4.10 Operator precedence and associativity

Theory

When multiple operators appear in one expression, **precedence** determines which operator is evaluated first. **Associativity** determines order when operators have equal precedence (left-to-right or right-to-left).

Precedence (high to low, summary):

1. `() [] -> .` — parentheses, subscript, member
2. `++ --` (prefix), `! ~ *` (deref) `&` (address) `sizeof` — unary
3. `*` `/` `%` — multiplicative
4. `+` `-` — additive
5. `<<` `>>` — shift
6. `<` `<=` `>` `>=` — relational
7. `==` `!=` — equality
8. `&` — bitwise AND
9. `^` — bitwise XOR
10. `|` — bitwise OR
11. `&&` — logical AND
12. `||` — logical OR
13. `?:` — conditional
14. `=` `+=` `-=` etc. — assignment (right-to-left)
15. `,` — comma (left-to-right)

Use parentheses to override default order and improve readability.

Important Points

- **Precedence** = priority order of operators
- **Associativity** = same-precedence evaluation direction
- Unary, multiplicative, additive, relational, logical, assignment, comma
- Assignment is **right-associative**
- Most binary operators: **left-associative**
- Use **()** when unsure — clarifies intent

For Exam

Operator precedence determines evaluation order in expressions. Higher precedence operators evaluate first: parentheses, then unary, then `*/%`, then `+,-, ,`, then relational, then logical, then conditional, then assignment, then comma. Associativity is the direction for equal precedence — assignment is right-to-left, most others left-to-right. Parentheses override default precedence.

1.4.11 Expressions

Theory

An **expression** is a combination of operands and operators that evaluates to a single value. Every expression has a **type** (int, float, etc.) matching its result.

Examples:

- `a + b` — arithmetic expression, type depends on operands
- `x > 0` — relational expression, result int (0 or 1)
- `a = b + c` — assignment expression (assigns and yields assigned value)
- `n++` — increment expression

Statements vs expressions: A statement is a complete instruction ending with `;`. An expression becomes a statement when followed by semicolon: `x = 5;` or even `x + y;` (valid but useless).

Expressions can be **nested**: `(a + b) * (c - d)`. The inner expressions are evaluated according to precedence. Side effects (like `++` in expression) occur during evaluation order.

Important Points

- Expression = operands + operators → single value
- Has a **type** (int, float, etc.)
- Expression + `;` = expression statement
- Can be nested: `(a+b)*(c-d)`
- Assignment expression has value (right-hand side)
- Understand precedence for correct evaluation

For Exam

An expression combines operands and operators to produce a value. Examples include `a+b` (arithmetic), `x>0` (relational), and `a=b+c` (assignment). Every expression has a type. Adding a semicolon makes it a statement. Expressions can be nested, and evaluation follows operator precedence rules.

1.4.12 Type conversion

Theory

Type conversion (casting) changes a value from one data type to another. C performs conversions in two ways:

Implicit conversion (coercion): Compiler automatically converts smaller/lower types to larger/higher types in mixed expressions. Rules: char/short promoted to int; int

promoted to float when mixed with float; float promoted to double. Example: `int a = 5; float b = 2.0; float c = a + b;` — a becomes float temporarily.

Explicit conversion (cast): Programmer forces conversion using cast syntax: `(type) expression`. Example: `(float)9 / 2` gives 4.5; without cast, `9/2` gives 4.

Casting is useful for integer division correction, pointer type changes, and function arguments. Be careful of **data loss** when converting from larger to smaller type (e.g., float to int truncates decimal).

Important Points

- **Implicit:** automatic promotion (int → float in mixed expr)
- **Explicit:** cast syntax (type)expr
- `(float)9/2 = 4.5; 9/2 = 4`
- char, short promoted to int in expressions
- Narrowing conversion may lose data
- Cast needed for integer division fix

For Exam

Type conversion changes a value from one type to another. Implicit conversion is done automatically by the compiler (e.g., int promoted to float in mixed expressions). Explicit conversion uses cast syntax: (type)expression, e.g., `(float)9/2` gives 4.5. Converting from larger to smaller types may cause data loss.

Block 2: I/O, Control Structures, Arrays and Pointers

Unit 1: Input Output Statements

2.1.1 Introduction to I/O

Theory

Input/Output (I/O) operations allow a C program to communicate with the outside world — typically the user via keyboard and screen (console I/O), or with files on disk (file I/O, covered in Block 4). Without I/O, a program could compute results but could not receive data or display them.

C treats all input and output as **streams of bytes**. The standard library `<stdio.h>` provides functions for console I/O. I/O functions are categorised as **formatted** (data converted according to format specifiers like %d) and **unformatted** (raw character or string transfer without format conversion).

Console I/O makes programs interactive — the user supplies input values and the program displays computed results. Proper use of format specifiers and the address-of operator (&) in scanf is essential to avoid runtime errors.

Important Points

- I/O = communication between program and user/files
- Console I/O via **stdio.h**
- Two categories: **formatted** and **unformatted**
- Formatted: scanf, printf (with % specifiers)
- Unformatted: getchar, putchar, gets, puts
- Streams of bytes model

For Exam

Input/Output in C enables communication between the program and the user or files. Console I/O uses functions from stdio.h. I/O is classified as formatted (using format specifiers like %d with scanf and printf) and unformatted (character/string functions like getchar and putchar). Formatted I/O converts data between internal representation and text.

2.1.2 Unformatted I/O functions

Theory

Unformatted I/O functions transfer data without format string conversion — typically one character or one string at a time.

Input functions:

- `getchar()` — reads one character from stdin (keyboard). Returns int (ASCII value or EOF).
- `gets(str)` — reads a string until newline; **deprecated/unsafe** (no bounds check) — prefer `fgets()` in practice, but SLM covers gets.

Output functions:

- `putchar(c)` — writes one character to stdout (screen).
- `puts(str)` — writes a string followed by automatic newline.

Unformatted I/O is simpler but less flexible than formatted I/O. Useful for character-by-character processing, menu systems, and simple string input. All require `#include <stdio.h>`.

Important Points

- **getchar()** — read one character
- **putchar(c)** — display one character
- **gets(str)** — read string (including spaces); unsafe
- **puts(str)** — print string + newline
- No format specifiers used
- Return values: getchar returns int/EOF

For Exam

Unformatted I/O functions transfer data without format conversion. `getchar()` reads one character; `putchar(c)` displays one character. `gets()` reads a string until newline (handles spaces); `puts()` prints a string with automatic newline. These functions are declared in `stdio.h` and are simpler than formatted I/O.

2.1.3 Formatted I/O functions

Theory

Formatted I/O uses format strings to control how data is converted between internal binary form and readable text.

`printf(format, arg1, arg2, ...)` — formatted output to screen. Format string contains literal text and conversion specifiers (`%d`, `%f`, etc.). Returns number of characters printed.

`scanf(format, &arg1, &arg2, ...)` — formatted input from keyboard. Requires **address-of operator (&)** before variable names (except arrays/strings which are already addresses). Reads input according to format specifiers.

Common mistakes: forgetting `&` in `scanf` (causes crash); mismatch between format specifier and variable type; leaving spaces in format that must match input exactly.

Both functions are declared in `<stdio.h>` and are the most frequently used I/O functions in C.

Important Points

- **`printf(format, args)`** — formatted output; no `&`
- **`scanf(format, &args)`** — formatted input; **& required** for variables
- Format string controls conversion
- Mismatch type ↔ specifier → undefined behaviour
- `scanf` stops at whitespace for `%s`
- Both in `stdio.h`

For Exam

Formatted I/O uses format strings for conversion. `printf(format, variables)` displays output — no `&` needed. `scanf(format, &variables)` reads input — `&` is mandatory before variable names (except strings). Format specifiers like `%d` for `int` and `%f` for `float` must match variable types. Both functions are in `stdio.h`.

```
#include<stdio.h>
int main() {
    int a, b, sum;
    printf("Enter two numbers: ");
    scanf("%d%d", &a, &b);
    sum = a + b;
    printf("Sum =%d\n", sum);
    return 0;
}
```

2.1.4 Escape sequences and format specifiers

Theory

Escape sequences begin with backslash `\` and represent special non-printable characters in strings and character constants:

Sequence	Meaning
	Newline (line break)
	Horizontal tab
	Carriage return
	Backspace
<code>\</code>	Backslash
<code>'</code>	Single quote
<code>"</code>	Double quote
<code>\0</code>	Null character

Format specifiers in `printf/scanf`:

Specifier	Type
<code>%d, %i</code>	<code>int</code>
<code>%u</code>	unsigned <code>int</code>
<code>%f</code>	<code>float/double</code>
<code>%c</code>	<code>char</code>

Specifier	Type
%s	string
%x	hexadecimal
%o	octal
%p	pointer address
%%	literal %

Width and precision: `%5d` (width 5), `%-5d` (left align), `%06d` (zero pad), `%6.2f` (width 6, 2 decimals).

Important Points

- Escape: newline, `ab`, `\` backslash, `\0` null
- `%d` int, `%f` float, `%c` char, `%s` string
- `%x` hex, `%o` octal, `%%` literal percent
- Width: `%5d`; precision: `%6.2f`
- `%-` left align; `%0` zero fill
- Match specifier to variable type

For Exam

Escape sequences represent special characters: (newline), tab), `\` (backslash), `\0` (null). Format specifiers include `%d` (int), `%f` (float), `%c` (char), `%s` (string), `%x` (hex), and `%o` (octal). Width and precision can be specified: `%5d` sets minimum width 5; `%6.2f` prints float with 2 decimal places in width 6.

Unit 2: Control Structures and Looping

2.2.1 Introduction to control structures

Theory

Control structures determine the order in which statements execute. By default, C executes sequentially — top to bottom. Control structures alter this flow.

Three fundamental types:

1. **Sequential** — statements execute one after another (default).
2. **Selection (decision)** — choose between paths based on condition (if, switch).
3. **Repetition (iteration/loop)** — repeat block while condition holds (for, while, do-while).

Additionally, **jump statements** (break, continue, goto) transfer control abruptly.

Structured programming emphasises using these control structures instead of unstructured goto jumps. Proper control structure selection makes programs readable, maintainable, and less error-prone. Every algorithm maps to combinations of sequence, selection, and iteration.

Important Points

- Three types: **sequential**, **selection**, **repetition**
- **Sequential** = **default top-to-bottom**
- **Selection**: **if**, **switch**
- **Repetition**: **for**, **while**, **do-while**
- **Jump**: **break**, **continue**, **goto**
- Structured programming avoids excessive goto

For Exam

Control structures govern program flow. Sequential control executes statements in order. Selection control (if, switch) chooses between paths based on conditions. Repetition control (for, while, do-while) repeats code. Jump statements (break, continue, goto) alter flow. Structured programming uses sequence, selection, and iteration instead of unstructured jumps.

2.2.2 Sequential control

Theory

Sequential control is the default execution model in C — statements execute in the order they appear, one after another, from top to bottom within a block. Each statement completes before the next begins (except when control structures redirect flow).

Example sequence:

```
statement1;
statement2;
statement3;
```

After statement1 executes, statement2 runs, then statement3. No branching or repetition occurs.

Sequential control forms the backbone of all programs. Even complex programs consist of sequential blocks inside functions, loops, and conditional branches. Variables retain values between sequential statements unless modified.

Important Points

- Default execution: top to bottom
- Each statement completes before next
- No branching in pure sequence
- Building block inside functions and loops
- Order matters when statements depend on prior results

For Exam

Sequential control is the default execution order in C where statements run one after another from top to bottom. Each statement executes completely before the next begins. Sequential execution forms the basis of all programs, even when combined with selection and repetition structures.

2.2.3 Selection control — if statement

Theory

The **if statement** executes a block of code only when a condition is true (non-zero):

```
if (condition)
    statement; // or { block }
```

If condition evaluates to non-zero (true), the statement executes; if zero (false), it is skipped. Curly braces `{ }` group multiple statements into one block.

Conditions typically use relational and logical operators: `if (marks >= 40)`, `if (age >= 18 && hasID)`.

The if statement provides one-way selection — action occurs only when condition is true; otherwise execution continues after the if block. It is the simplest decision-making construct in C.

Important Points

- Syntax: `if (condition) statement;`
- Condition true (non-zero) → execute body
- Condition false (zero) → skip body
- Use `{ }` for multiple statements
- Single statement without braces allowed (indent for clarity)
- Most basic selection structure

For Exam

The if statement executes a statement or block only when the condition is true (non-zero). Syntax: if (condition) statement;. If the condition is false (zero), the statement is skipped. Curly braces group multiple statements. It is used for one-way decision making.

2.2.4 if-else and nested if

Theory

if-else provides two-way selection — one block when true, another when false:

```
if (condition)
    statement1;
else
    statement2;
```

Exactly one branch executes. The else binds to the nearest unmatched if.

Nested if: An if inside another if (or else). Used for multi-level decisions:

```
if (x > 0)
    if (x > 100)
        printf("Large positive");
    else
        printf("Small positive");
```

Dangling else problem: Always use braces to clarify which else belongs to which if. Nested if can replace else-if ladders for complex logic but may reduce readability.

Important Points

- if-else: two branches, exactly one executes
- else binds to **nearest** if
- **Nested if:** if inside if/else
- Use { } to avoid dangling else confusion
- Can handle multi-level decisions
- else-if ladder alternative for many conditions

For Exam

The if-else statement provides two-way selection: if condition is true, the if block executes; otherwise the else block executes. Nested if places one if statement inside

another for multi-level decisions. The else always pairs with the nearest if. Braces should be used for clarity when nesting.

2.2.5 else-if ladder

Theory

An **else-if ladder** (else if chain) tests multiple conditions sequentially:

```
if (condition1)
    block1;
else if (condition2)
    block2;
else if (condition3)
    block3;
else
    default_block;
```

Conditions are tested top to bottom. The **first true** condition's block executes, and remaining conditions are skipped. The final else is optional — handles no-match case.

Use when comparing a variable against several ranges or categories (grade calculation, menu selection with ranges). Differs from nested if in flat structure. If no condition matches and no else exists, no block executes.

Important Points

- Chain: if → else if → else if → else
- First **true** condition wins; rest skipped
- Final **else** optional (default case)
- Good for grade ranges, multi-category decisions
- Only **one** block executes
- Order of conditions matters

For Exam

An else-if ladder tests multiple conditions in sequence using if, else if, and optional else. Conditions are checked from top to bottom and the first true condition's block executes. Remaining conditions are skipped. The final else handles the case when no condition is true. Only one block executes.

2.2.6 switch statement

Theory

The **switch** statement selects among multiple branches based on the value of an integer or character expression:

```
switch (expression) {  
    case constant1: statements; break;  
    case constant2: statements; break;  
    default: statements;  
}
```

Rules:

- Expression must be **integer or char** type (not float, string).
- Each **case** label must be a **constant** (not variable).
- **break** exits switch; without it, execution **falls through** to next case.
- **default** is optional — handles unmatched values.

Switch is cleaner than long else-if chains when comparing one variable against many constant values (menu options 1, 2, 3).

Important Points

- switch(expr) with case labels
- Expression: **int or char** only
- case values must be **constants**
- **break** prevents fall-through
- **default** — optional catch-all
- Fall-through without break: executes next cases
- Ideal for menu-driven programs

For Exam

The switch statement selects one of many branches based on an integer or character expression. Each case label must be a constant. break exits the switch; without break, execution falls through to the next case. default handles unmatched values. Switch is efficient for multi-way branching on a single variable.

```
switch(choice) {  
    case 1: printf("Add"); break;  
    case 2: printf("Subtract"); break;  
    case 3: printf("Multiply"); break;  
}
```

```
default: printf("Invalid");  
}
```

2.2.7 Jump statements

Theory

Jump statements transfer program control unconditionally or conditionally to another point:

- **break** — exits innermost loop or switch immediately.
- **continue** — skips remaining statements in current loop iteration; proceeds to next iteration.
- **goto label** — jumps to labelled statement anywhere in same function.

Jump statements alter normal sequential/loop flow. **break** and **continue** are structured and widely used. **goto** is discouraged in structured programming because it creates spaghetti code that is hard to follow and debug — use **break/continue** or functions instead.

break in nested loops exits only the innermost loop. A labelled **break** requires **goto** (rare) or restructuring code.

Important Points

- **break** — exit loop or switch
- **continue** — skip to next loop iteration
- **goto label** — unconditional jump (avoid)
- **break** exits **innermost** enclosing loop/switch
- **continue** affects **current iteration only**
- Prefer structured control over **goto**

For Exam

Jump statements alter program flow. **break** exits the innermost loop or switch immediately. **continue** skips the rest of the current loop iteration and starts the next. **goto** jumps to a label but is discouraged in structured programming. **break** and **continue** are commonly used; **goto** should be avoided.

2.2.8 for loop

Theory

The **for loop** is an entry-controlled loop (condition checked before body) ideal when the number of iterations is known:

```
for (initialization; condition; update)
    statement;
```

Execution order:

1. Initialization (once)
2. Check condition — if false, exit loop
3. Execute body
4. Execute update expression
5. Go to step 2

Example: `for (int i = 1; i <= 10; i++) printf("%d ", i);` prints 1 to 10.

All three parts are optional: `for(;;)` creates infinite loop (exit with `break`). Multiple variables can be initialized/updated using comma operator: `for (i=0, j=10; i<j; i++, j--)`.

Semicolon after `for` header creates empty loop body — often a bug.

Important Points

- Syntax: `for(init; condition; update) body;`
- **Entry-controlled** — may run 0 times
- Init once; condition before each iteration
- All three parts optional
- Common for counting loops
- `for(;;)` = infinite loop
- Semicolon after `for()` = empty body (bug)

For Exam

The for loop is an entry-controlled loop with three parts: initialization, condition, and update. The condition is checked before each iteration, so the loop may run zero times. Example: `for(i=1; i<=10; i++)` repeats 10 times. It is ideal when the number of iterations is known in advance.

```
for (int i = 1; i <= 10; i++)
    printf("%d ", i);
```

2.2.9 while loop

Theory

The **while loop** is entry-controlled — the condition is tested **before** each iteration:

```
while (condition)
    statement;
```

If condition is initially false, the body **never executes** (zero iterations minimum). Used when iterations depend on a condition rather than a fixed count.

Example — read until sentinel:

```
while (num != 0) {
    sum += num;
    scanf("%d", &num);
}
```

The condition typically changes inside the loop body; otherwise infinite loop occurs. while is more flexible than for when the number of iterations is unknown beforehand.

Important Points

- **Entry-controlled** — condition before body
- May run **0 times** if condition initially false
- Use when iterations depend on condition
- Must update condition variable in body
- while(1) or while(true) with break inside = infinite loop pattern
- More general than for for unknown count

For Exam

The while loop is entry-controlled: the condition is checked before each iteration. If the condition is false initially, the body never executes. Syntax: while(condition) statement;. It is used when the number of iterations is not known in advance, such as reading input until a sentinel value.

2.2.10 do-while loop

Theory

The **do-while loop** is exit-controlled — the body executes **at least once** because the condition is checked **after** the body:

```
do {  
    statement;  
} while (condition);
```

Note the semicolon after while(condition). Use when you need guaranteed first execution — menu systems ("show menu, process choice, repeat if invalid"), input validation.

Comparison: while checks first (0+ iterations); do-while checks last (1+ iterations). Choose do-while when the loop body must run before knowing whether to continue.

Important Points

- **Exit-controlled** — condition after body
- Runs **at least once**
- Syntax: `do { } while (condition);`
- Semicolon required after while(condition)
- Use for menus, input validation
- while: 0+ times; do-while: 1+ times

For Exam

The do-while loop is exit-controlled: the body executes first, then the condition is checked. It always runs at least once. Syntax: `do { statements; } while (condition);`. Use it when the loop body must execute before testing the condition, such as in menu-driven programs.

2.2.11 break, continue and goto

Theory

These three jump statements serve distinct purposes in loops and switch:

break: Immediately terminates the innermost enclosing loop or switch. Control passes to the statement after the loop/switch. In switch, break prevents fall-through to next case.

continue: Skips remaining statements in the current iteration of a loop and jumps to the condition check (while/for) or condition test after body (do-while). Does not exit the loop entirely.

goto label: Transfers control to the statement marked by label:. Valid but discouraged — makes programs hard to read, maintain, and verify. Acceptable in rare cases (error cleanup in C systems code).

Example: skip even numbers — `if (i % 2 == 0) continue;`

Important Points

- **break** — exit loop/switch completely
- **continue** — skip to next iteration
- **goto** — jump to label (avoid in normal code)
- break in switch prevents case fall-through
- continue only skips current iteration
- break exits innermost loop only

For Exam

break exits the innermost loop or switch immediately. continue skips the remaining statements in the current loop iteration and proceeds to the next iteration. goto transfers control to a labelled statement but is discouraged in structured programming. break is essential in switch to prevent fall-through.

2.2.12 Nested loops

Theory

A **nested loop** is a loop inside another loop. The inner loop completes all its iterations for each iteration of the outer loop.

Example — multiplication table:

```
for (i = 1; i <= 3; i++)
    for (j = 1; j <= 3; j++)
        printf("%d ", i * j);
```

If outer runs n times and inner runs m times per outer iteration, total inner executions = $n \times m$.

Common uses: 2D array processing, pattern printing, matrix operations, combinations. break in nested loop exits only the inner loop — to exit outer loop, use flag variable or goto (or refactor to function with return).

Performance: deeply nested loops can be slow for large n .

Important Points

- Loop inside another loop
- Inner completes fully for each outer iteration
- Total iterations = product of loop counts

- Used for matrices, patterns, tables
- break exits **innermost** loop only
- Outer controls rows; inner controls columns (typical)

For Exam

A nested loop contains one loop inside another. The inner loop runs completely for each iteration of the outer loop. Total iterations equal the product of individual loop counts. Nested loops are used for 2D array processing, pattern printing, and multiplication tables. break exits only the innermost loop.

Unit 3: Arrays and Strings

2.3.1 One-dimensional arrays

Theory

An **array** is a collection of elements of the **same data type** stored in **contiguous memory** locations. A one-dimensional array is a linear sequence.

Declaration: `data_type array_name[size];`

Example: `int marks[5];` — holds 5 integers.

Initialization: `int arr[5] = {10, 20, 30, 40, 50};` or `int arr[] = {1, 2, 3};` (size inferred).

Index starts at **0**; last index = size - 1. Access: `arr[i]`. C does **not** check bounds — accessing `arr[10]` when size is 5 causes undefined behaviour.

Array name represents the **address of first element** (constant pointer). Size cannot be changed after declaration (static arrays).

Important Points

- Same type elements in contiguous memory
- Declaration: type name[size];
- Index: **0** to **size-1**
- No bounds checking in C
- Array name = address of first element
- `int arr[5] = {1,2,3,4,5};`

For Exam

A one-dimensional array stores multiple elements of the same type in contiguous memory. Declared as type name[size]. Index starts at 0; last index is size-1. Elements accessed as `arr[i]`. C does not check array bounds. The array name represents the address of the first element.

2.3.2 Two-dimensional arrays

Theory

A **two-dimensional array** is an array of arrays — useful for matrices, tables, and grids.

Declaration: `data_type name[rows][cols];`

Example: `int matrix[3][4];` — 3 rows, 4 columns.

Initialization: `int m[2][3] = {{1,2,3}, {4,5,6}};`

Access: `matrix[i][j]` where i is row, j is column (both zero-based).

Memory is stored **row-wise** (row-major order): first row elements, then second row, etc. Total elements = rows × columns.

Can pass to functions with row and column sizes. Pointer equivalent: `*(*(matrix + i) + j)` equals `matrix[i][j]`.

Important Points

- Declaration: `type name[rows][cols];`
- Access: `arr[i][j]` — i=row, j=column
- Stored **row-major** in memory
- Total elements = rows × cols
- Used for matrices, tables
- Initialize with nested braces `{{},{}}`

For Exam

A two-dimensional array represents a matrix with rows and columns. Declared as `type name[rows][cols]`. Accessed as `arr[i][j]` where i is row index and j is column index. Elements are stored row-wise in memory. Total elements equal rows multiplied by columns.

2.3.3 Strings

Theory

In C, a **string** is a one-dimensional array of characters terminated by the **null character** `'\0'` (ASCII 0). There is no separate string data type — strings are char arrays.

Declaration: `char str[20];` — holds up to 19 characters plus `'\0'`.

String literal: `char greeting[] = "Hello";` — compiler adds `'\0'` automatically.

The null character marks the end of the string. Functions like `strlen` stop at `'\0'`. Without null terminator, string functions read past array bounds.

Single quotes `'A'` = one character; double quotes `"A"` = string of two chars: `'A'` and `'\0'`.

Important Points

- String = char array ending with `'\0'`
- No separate string type in C
- `char str[20];` holds 19 chars + null
- Compiler adds `\0` to string literals
- `'A'` = char; `"A"` = string
- `strlen` counts until `\0` (excludes `\0`)

For Exam

In C, a string is a character array terminated by the null character `'\0'`. Declared as `char str[size]`. String literals in double quotes automatically include the null terminator. The null character marks the end of the string. Single quotes denote a character; double quotes denote a string.

2.3.4 String input methods

Theory

Several methods read strings from input:

`scanf("%s", str)` — reads until whitespace; **cannot** read strings with spaces. No & needed (str is array name = address).

`gets(str)` — reads entire line including spaces until Enter. **Unsafe** — no buffer size limit. SLM covers it; modern practice uses `fgets`.

`scanf("%[^\n]s", str)` — reads until newline; handles spaces.

`fgets(str, size, stdin)` — safe; reads at most size-1 characters.

After input, strings must remain null-terminated for string functions to work correctly. Always ensure array is large enough for input plus `'\0'`.

Important Points

- `scanf("%s")` — stops at whitespace; no spaces
- `gets()` — full line with spaces; unsafe
- `scanf("%[^\n]s")` — line with spaces
- `fgets()` — safe alternative (not always in SLM)

- No & before array name in scanf
- Ensure room for \0 terminator

For Exam

String input methods include `scanf("%s", str)` which reads until whitespace and cannot handle spaces, `gets(str)` which reads a full line including spaces but is unsafe due to no bounds checking, and `scanf("%[^\n]s", str)` which reads until newline. The array name is used without & in scanf since it is already an address.

2.3.5 String handling functions

Theory

The `<string.h>` header provides standard string functions:

Function	Purpose	Returns
<code>strlen(s)</code>	Length of string	int (excl. \0)
<code>strcpy(dest, src)</code>	Copy src to dest	dest
<code>strncpy(dest, src, n)</code>	Copy n chars	dest
<code>strcmp(s1, s2)</code>	Compare strings	0=equal, <0, >0
<code>strcat(s1, s2)</code>	Append s2 to s1	s1
<code>strncat(s1, s2, n)</code>	Append n chars	s1

strcmp compares lexicographically (ASCII order), not alphabetically locale-aware. Returns 0 if equal.

strcpy/strcat require dest large enough — buffer overflow if too small. All assume null-terminated strings.

Important Points

- Header: **#include <string.h>**
- `strlen` — length excluding \0
- `strcpy` — copy; dest must be large enough
- `strcmp` — 0 if equal; <0 if `s1<s2`; >0 if `s1>s2`
- `strcat` — append; dest must have space
- All require null-terminated strings

For Exam

String handling functions in `string.h` include: `strlen` (returns length excluding null), `strcpy` (copies source to destination), `strcmp` (compares strings, returns 0 if equal),

and strcat (appends source to destination). All functions require null-terminated strings. strcmp returns negative, zero, or positive for less than, equal, or greater.

2.3.6 Array of strings

Theory

An **array of strings** is a two-dimensional character array where each row holds one string:

```
char names[5][20]; // 5 strings, max 19 chars each
```

Or array of character pointers:

```
char *names[] = {"Alice", "Bob", "Charlie"};
```

Each row is a null-terminated string. Access: `names[i]` refers to i-th string; `names[i][j]` is j-th character of i-th string.

Used for lists of names, menu options, dictionary words. When passing to functions, pass row count and column size for 2D char array.

Important Points

- 2D char array: `char names[5][20];`
- Each row = one string with `\0`
- Access: `names[i]` = i-th string
- Pointer array: `char *arr[] = {"a","b"};`
- Used for name lists, menus
- Pass rows and max length to functions

For Exam

An array of strings is typically a two-dimensional character array `char name[count][length]` where each row stores one null-terminated string. `names[i]` accesses the i-th string. Alternatively, an array of char pointers can point to string literals. Used for storing multiple names or words.

Unit 4: Pointers and Dynamic Memory Allocation

2.4.1 Introduction to pointers

Theory

A **pointer** is a variable that stores a **memory address** — the location of another variable or data.

Declaration: `data_type *pointer_name;`

Example: `int *p;` — p can hold address of an int.

Address-of operator (&): `p = &i;` — p stores address of i.

****Dereference/indirection (*):** `*p` — value stored at address p.

Pointers enable dynamic memory, efficient array/string handling, function parameter modification, and data structures. Uninitialized pointers contain garbage addresses — dereferencing causes crash. Always initialize pointers before use.

Important Points

- Pointer stores **memory address**
- Declaration: type *ptr;
- **&** — address-of; ******** — dereference (value at address)
- `int p; p = &i; p = value of i`
- Uninitialized pointer → dangerous
- Enables dynamic memory and call by reference

For Exam

A pointer is a variable that stores the memory address of another variable. Declared as type *ptr*. *The & operator gets the address of a variable; the operator accesses the value at that address.* For example, `p = &i` makes p point to i, and `*p` gives the value of i.

2.4.2 Pointer arithmetic and arrays

Theory

Pointers support **arithmetic** — adding/subtracting integers moves the pointer by multiples of the pointed-to type's size:

- `p + 1` moves to next int (typically 4 bytes forward)
- `p - 1` moves to previous element
- `p2 - p1` = number of elements between them

Array-pointer relationship: Array name decays to pointer to first element. `arr[i]` equals `*(arr + i)`. Can traverse array with pointer:

```
int arr[5] = {1,2,3,4,5};
int *p = arr;
```

```
for (int i = 0; i < 5; i++)
    printf("%d ", *(p + i));
```

Pointer arithmetic is valid only within the same array object bounds.

Important Points

- `p+1` moves by **sizeof(type)** bytes
- `arr[i]` same as `*(arr+i)`
- Array name = constant pointer to first element
- Pointer subtraction gives element count
- Valid only within same array
- Traverse array with pointer increment

For Exam

Pointer arithmetic adds or subtracts multiples of the data type size. `arr[i]` is equivalent to `*(arr+i)`. The array name represents the address of the first element. Incrementing a pointer moves to the next element of its type. Pointer subtraction between array elements gives the number of elements between them.

2.4.3 Dynamic memory allocation

Theory

Static memory (regular variables, fixed arrays) is allocated at compile time on the stack or in data segment. **Dynamic memory** is allocated at **runtime** from the **heap** using `<stdlib.h>` functions:

Function	Purpose	Initializes?
<code>malloc(size)</code>	Allocates size bytes	No (garbage)
<code>calloc(n, size)</code>	Allocates <code>n*size</code> bytes	Yes (zero)
<code>realloc(ptr, newsize)</code>	Resize block	Preserves data
<code>free(ptr)</code>	Release memory	—

Example: `int *arr = (int*)malloc(5 * sizeof(int));`

Always check for **NULL** return (allocation failure). Always **free()** allocated memory to prevent **memory leak**. Do not use memory after free.

Important Points

- `malloc(size)` — allocate; uninitialized
- `calloc(n, size)` — allocate; zero-initialized

- `realloc(ptr, size)` — resize
- `free(ptr)` — release memory
- Returns `void*` — cast to needed type
- Check `NULL`; always free after use
- Memory leak if free omitted

For Exam

Dynamic memory is allocated at runtime from the heap using `malloc` (allocates bytes, uninitialized), `calloc` (allocates n elements of given size, initialized to zero), `realloc` (resizes existing block), and `free` (releases memory). `malloc` returns void pointer requiring cast. Always check for `NULL` and call `free` to avoid memory leaks.

```
int *arr = (int*)malloc(5 * sizeof(int));
if (arr == NULL) { printf("Failed"); exit(1); }
arr[0] = 10;
free(arr);
```

2.4.4 Pointers and strings

Theory

Strings and pointers are closely related. A char pointer can point to a string literal or character array:

```
char *ptr = "Hello";
```

String literal stored in read-only memory; ptr points to first character. Traverse with pointer:

```
while (*ptr != '\0') {
    putchar(*ptr);
    ptr++;
}
```

Alternatively: `char str[] = "Hello"; char *p = str;`

Difference: `char *p = "text"` — pointer to literal; `char arr[] = "text"` — modifiable array copy. String functions often return char pointers. Array of char pointers stores multiple strings efficiently.

Important Points

- `char *ptr = "Hello"` — pointer to string literal
- Traverse: increment `ptr` until `*ptr == '\0'`
- String literal may be read-only
- `char arr[] = copy; char *p = pointer`
- Pointer arithmetic for character-by-character access
- Used with dynamic strings and function returns

For Exam

A character pointer can point to a string literal: `char ptr = "Hello"`. Strings can be traversed by incrementing the pointer until the null character is found. The expression `ptr` accesses each character. Pointers enable efficient string processing and dynamic string allocation.

Block 3: Functions, Structures and Union

Unit 1: Functions

3.1.1 Modular programming

Theory

Modular programming is a design approach that divides a large program into smaller, independent modules called **functions**. Each module performs a specific, well-defined task. This mirrors the divide-and-conquer strategy used in problem-solving.

Benefits of modular programming:

- **Reduced complexity** — each function handles one job
- **Easier debugging** — test functions independently
- **Code reusability** — same function called multiple times
- **Easier maintenance** — changes localised to one module
- **Team development** — different programmers work on different modules
- **Top-down design** — main controls high-level flow; sub-functions handle details

In C, every program uses modular structure through `main()` and user-defined or library functions. A well-designed C program reads like an outline in `main()`, with details delegated to functions.

Important Points

- Divide program into smaller **functions/modules**
- Each module = one specific task

- Benefits: simpler debugging, reuse, maintenance
- Supports top-down design
- `main()` coordinates; functions implement details
- Foundation of structured programming in C

For Exam

Modular programming divides a large program into smaller independent functions, each performing a specific task. Advantages include reduced complexity, easier debugging, code reusability, easier maintenance, and support for team development. In C, `main()` and user-defined functions implement modular design.

3.1.2 Built-in functions

Theory

Built-in functions (also called library functions or predefined functions) are functions provided by C standard libraries and available through header files. The programmer does not write their code — only calls them.

Examples:

- `<stdio.h>`: `printf`, `scanf`, `getchar`, `putchar`, `gets`, `puts`
- `<string.h>`: `strlen`, `strcpy`, `strcmp`, `strcat`
- `<stdlib.h>`: `malloc`, `calloc`, `free`, `exit`, `atoi`
- `<math.h>`: `sqrt`, `pow`, `sin`, `cos`

To use a built-in function, include the appropriate header with `#include <header.h>`. The linker connects calls to library implementations during linking.

Built-in functions are tested, optimised, and portable — using them saves development time and reduces errors.

Important Points

- Also called **library/predefined** functions
- Provided in header files (`stdio.h`, `string.h`, etc.)
- **#include** required before use
- Examples: `printf`, `scanf`, `strlen`, `malloc`
- Linker connects to library at link time
- Programmer calls but does not define them

For Exam

Built-in (library) functions are predefined in C standard libraries and declared in header files. Examples include `printf` and `scanf` in `stdio.h`, `strlen` and `strcpy` in

string.h, and malloc and free in stdlib.h. They are used by including the header file and calling the function. The linker resolves them during linking.

3.1.3 User-defined functions

Theory

User-defined functions are functions written by the programmer to perform tasks specific to the application. They extend C's capabilities beyond built-in libraries.

A function has:

- **Return type** — data type of value returned (or void for none)
- **Function name** — identifier following naming rules
- **Parameter list** — zero or more parameters in parentheses
- **Function body** — statements in `{ }` performing the task

Example concept: a function `int square(int n)` returns $n \times n$. User-defined functions allow custom logic — calculating tax, validating input, printing formatted reports — tailored to program requirements.

Every C program must have at least `main()` — a user-defined function that serves as entry point (though technically the runtime calls it).

Important Points

- Written by programmer for specific tasks
- Components: return type, name, parameters, body
- **void** return type = no return value
- Can have zero or more parameters
- `main()` is mandatory user-defined function
- Extend beyond standard library capabilities

For Exam

User-defined functions are created by the programmer to perform application-specific tasks. A function has a return type, name, parameter list, and body enclosed in braces. `void` indicates no return value. Every C program must contain `main()` as the entry point function.

3.1.4 Function declaration

Theory

A **function declaration** (also called function prototype) tells the compiler about a function's name, return type, and parameters **before** it is defined or called. This allows the function to be defined after `main()` or in another file.

Syntax: `return_type function_name(parameter_types);`

Examples:

```
int add(int, int);
void display(void);
float average(int, float);
```

Parameter names in prototype are optional — types are required: `int add(int a, int b);` equals `int add(int, int);`.

Without declaration, C assumes old rules (int return, untyped parameters) — causes errors in modern compilers. Prototypes enable type checking at call site.

Important Points

- Also called **function prototype**
- Declares name, return type, parameter types
- Ends with semicolon; no body
- Allows function definition after main()
- Parameter names optional in prototype
- Enables compiler type checking

For Exam

A function declaration (prototype) specifies the function's return type, name, and parameter types before its definition. Syntax: `return_type name(parameter_types);`. It ends with a semicolon and has no body. Prototypes allow functions to be defined after main() and enable the compiler to check function calls.

3.1.5 Function definition

Theory

A **function definition** provides the complete implementation — the actual code that executes when the function is called.

Syntax:

```
return_type function_name(parameter_list) {
    // local declarations
    // statements
    return value; // if not void
}
```

The definition must match the declaration in return type and parameter types. Parameters in definition are **formal parameters** — local variables receiving values from the caller.

If return type is not void, a `return` statement sends a value back. Execution stops at return — code after return in same block is unreachable. A function can have multiple return statements (different exit paths).

Important Points

- Contains actual function **body/code**
- Must match prototype in type and parameters
- Parameters = **formal parameters**
- **return** sends value back (unless void)
- Code after return is not executed
- Local variables declared inside body

For Exam

A function definition contains the complete implementation of the function including the body in braces. It must match the declaration in return type and parameter types. Formal parameters receive values from the caller. The return statement sends a value back to the calling function unless the return type is void.

3.1.6 Function call

Theory

A **function call** invokes a function to execute its code. When called, control transfers to the function; after completion, control returns to the statement after the call.

Syntax: `function_name(arguments);` or `variable = function_name(arguments);`

Arguments (actual parameters) must match formal parameters in number and type. The compiler converts call to passing values (call by value) or addresses (call by reference with pointers).

Example: `result = add(5, 7);` — calls add with arguments 5 and 7; return value stored in result.

Function call involves: save return address, allocate space for parameters/locals, execute body, return value, restore caller state. Recursive calls create multiple stack frames.

Important Points

- **function_name(args)** invokes function

- Arguments = **actual parameters**
- Must match prototype in number and type
- Control returns after function completes
- Return value can be used in expression
- Recursion: function calls itself

For Exam

A function call executes the function by writing its name followed by arguments in parentheses. Arguments (actual parameters) must match the formal parameters in number and type. Control transfers to the function and returns after execution. The return value can be assigned to a variable or used in an expression.

3.1.7 Examples of functions

Theory

Common function examples illustrate declaration, definition, and call working together:

Function returning int:

```
int factorial(int n) {
    int f = 1, i;
    for (i = 1; i <= n; i++)
        f *= i;
    return f;
}
```

void function (no return):

```
void greet(char name[]) {
    printf("Hello,%s!\n", name);
}
```

Function with multiple parameters:

```
float avg(int count, float sum) {
    return sum / count;
}
```

In main: `int f = factorial(5);` `greet("Ali");` `float a = avg(3, 270.0);`

These patterns — computation functions, output functions, utility functions — appear throughout C programs.

Important Points

- **int function** — returns computed value
- **void function** — performs action, no return
- Declare prototype before main if defined after
- Call from main or other functions
- Parameters pass data into function
- return provides output to caller

For Exam

Function examples include int functions that return computed values (like factorial), void functions that perform actions without returning values (like greet), and functions with multiple parameters (like average). Each requires declaration, definition, and call. The return statement passes results back to the caller.

```
#include<stdio.h>
int add(int, int);
int main() {
    int sum = add(5, 7);
    printf("Sum =%d", sum);
    return 0;
}
int add(int a, int b) {
    return a + b;
}
```

3.1.8 Nested functions

Theory

Nested functions in the SLM context means one function **calling another function** from within its body — not defining a function inside another (which standard C does not allow).

Example:

```
int square(int x) { return x * x; }
int sum_of_squares(int a, int b) {
```

```
    return square(a) + square(b); // nested call
}
```

main calls sum_of_squares, which calls square twice — creating a call hierarchy.

Call stack: main → sum_of_squares → square → return → square → return → return to main.

Design nested calls so each function has a single clear responsibility. Deep call chains can be hard to debug — balance modularity with simplicity.

Important Points

- **Nested function call** = function calls another function
- C does **not** allow function defined inside function
- Creates call hierarchy / call stack
- Each function should have one clear task
- main → funcA → funcB pattern
- Return unwinds the call stack

For Exam

Nested functions refer to one function calling another from within its body. Standard C does not allow defining a function inside another function. For example, main may call funcA which calls funcB. Each call adds a frame to the call stack, and returns unwind the stack in reverse order.

Unit 2: Recursion

3.2.1 Recursion

Theory

Recursion is a technique where a function **calls itself** directly or indirectly to solve a problem by breaking it into smaller instances of the same problem.

Every recursive function must have:

1. **Base case** — simple condition where function returns without recursing (stops recursion)
2. **Recursive case** — function calls itself with modified (smaller/simpler) argument

Example — factorial: $n! = n \times (n-1)!$; base: $0! = 1$.

```
int fact(int n) {
    if (n <= 1)
        return 1; // base case
}
```

```
}  
return n * fact(n - 1); // recursive case
```

Recursion uses **call stack** — each call waits for inner call to return. Without base case → **infinite recursion** → stack overflow crash.

Recursion suits mathematical definitions, tree traversal, and divide-and-conquer; iteration is often faster for simple loops.

Important Points

- Function **calls itself**
- Requires **base case** (stop condition)
- Requires **recursive case** (smaller problem)
- No base case → infinite recursion / stack overflow
- Examples: factorial, Fibonacci, sum of n
- Uses more memory than loops (stack frames)

For Exam

Recursion is when a function calls itself to solve a problem. It must have a base case that stops recursion and a recursive case that calls itself with a smaller problem. Example: factorial where $\text{fact}(n) = n * \text{fact}(n-1)$ and base case $\text{fact}(0) = 1$. Without a base case, infinite recursion causes stack overflow.

```
int fact(int n) {  
    if (n <= 1) return 1;  
    return n * fact(n - 1);  
}
```

3.2.2 Types of recursion and arrays in functions

Theory

Types of recursion:

Direct recursion: Function calls itself directly. Example: `fact(n)` calls `fact(n-1)`.

Indirect recursion: Function A calls function B, which calls function A back. Rare in introductory programming.

Linear recursion: One recursive call per invocation (factorial). **Tree recursion:** Multiple recursive calls (Fibonacci — inefficient without memoization).

Arrays in functions: Arrays are passed to functions by passing the array name (address). Size usually passed separately:

```
void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
}
```

Changes to array elements inside function **reflect in caller** — effectively call by reference for arrays. 2D arrays: pass with row and column dimensions: `void func(int a[][4], int rows)`.

Recursion with arrays: process element n-1 after processing first n-1 elements recursively.

Important Points

Recursion Type	Description
Direct	Function calls itself
Indirect	A calls B, B calls A
Linear	One recursive call
Tree	Multiple recursive calls

- Arrays passed by **array name** (address)
- Pass **size** separately
- Changes to elements visible in caller
- 2D array: specify column size in parameter
- Recursive array processing reduces size each call

For Exam

Types of recursion include direct (function calls itself) and indirect (A calls B which calls A). Arrays passed to functions use the array name without subscript; size must be passed separately. Modifications to array elements inside the function affect the original array. For 2D arrays, column size is required in the parameter declaration.

Unit 3: Call by Value and Call by Reference

3.3.1 Parameters — local and global variables

Theory

Variables in C have scope and lifetime determined by where they are declared.

Local variables: Declared inside a function or block. Visible only within that function/block. Created when block entered, destroyed when block exits. Default storage class: auto.

Global variables: Declared outside all functions (before main). Visible to all functions from declaration point to end of file. Exist for entire program execution. Default initialized to zero if not explicitly set.

Parameters (formal parameters): Local variables of the called function, receiving values from arguments.

Name hiding (shadowing): If local variable has same name as global, local takes precedence inside that function.

Global variables simplify sharing data but reduce modularity and make debugging harder — use sparingly.

Important Points

- **Local:** inside function; visible only there; destroyed on exit
- **Global:** outside functions; visible to all; lifetime = program
- **Formal parameters** = local to called function
- Local **shadows** global of same name
- Globals default to 0; auto locals = garbage
- Prefer locals; minimize globals

For Exam

Local variables are declared inside a function and are accessible only within that function. Global variables are declared outside all functions and are accessible throughout the program. If a local variable has the same name as a global variable, the local variable hides the global within that function. Formal parameters are local to the called function.

3.3.2 Call by value and call by reference

Theory

C passes arguments to functions primarily by **call by value** — a copy of the argument is passed to the formal parameter. Changes inside the function do not affect the original variable.

Call by value: `void func(int x)` — x is copy of argument. `func(a)` — changes to x do not change a.

Call by reference (simulated with pointers): Pass address using `&`; receive with pointer parameter. Changes through pointer affect original.

```
void swap(int *a, int *b) {
    int t = *a;
    *a = *b;
    *b = t;
}
// Call: swap(&x, &y);
```

Arrays are effectively call by reference (array name is address). Call by reference enables functions to return multiple results via parameters and is essential for swap, modify-in-place, and efficient large data handling.

Important Points

Method	Call syntax	Parameter	Original changed?
Call by value	func(a)	int x	No
Call by reference	func(&a)	int *x	Yes

- Default in C = **call by value**
- Reference simulated with **pointers**
- swap(&x, &y) classic example
- Arrays pass by address automatically
- Pointers allow multiple output values

For Exam

In call by value, a copy of the argument is passed to the function, so changes inside the function do not affect the original. In call by reference (using pointers), the address is passed using & and received as a pointer parameter, so changes affect the original. Example: swap(&x, &y) with void swap(int a, int b). Arrays are passed by address.

```
void swap(int *a, int *b) {
    int t = *a; *a = *b; *b = t;
}
int main() {
    int x = 2, y = 3;
    swap(&x, &y); // x=3, y=2
    return 0;
}
```

Unit 4: Structures and Union

3.4.1 Introduction to structures

Theory

A **structure** is a user-defined data type that groups related variables of **different types** under a single name. While arrays hold many values of the same type, structures hold a collection of logically related fields of various types.

Example concept: a Student record with name (char array), roll number (int), and marks (float) — kept together as one unit.

Structures model real-world entities: employee, book, date, point (x,y). They are essential for organizing complex data in C programs before object-oriented features.

The keyword `struct` defines the structure type; variables of structure type hold all members together in memory (with possible padding for alignment).

Important Points

- Groups **different types** under one name
- User-defined data type using **struct**
- Models real-world records (student, employee)
- Unlike array: mixed data types
- Members accessed by name
- Foundation for complex data organization

For Exam

A structure is a user-defined data type that groups variables of different types under one name using the struct keyword. Unlike arrays which store same-type elements, structures combine related fields like name, id, and marks into one record. Structures model real-world entities in programs.

3.4.2 Defining a structure

Theory

Structure definition syntax:

```
struct tag_name {  
    data_type member1;  
    data_type member2;  
    . . .  
};
```

Example:

```
struct student {  
    char name[20];  
    int roll_no;  
    float marks;  
};
```

After definition, declare variables:

```
struct student s1, s2;
```

Or define and declare together:

```
struct student {  
    char name[20];  
    int roll;  
} s1, s2;
```

Structure tag (student) creates a type name used with `struct` keyword (unless typedef used). Members can be any valid C types including arrays, pointers, and other structures.

Important Points

- Syntax: struct tag { members };
- Tag name identifies structure type
- Declare: struct tag var1, var2;
- Can define + declare in one step
- Members: any valid C types
- Semicolon after closing brace required

For Exam

A structure is defined using struct tag_name { member declarations };. Example: struct student { char name[20]; int roll_no; float marks };. Variables are declared as struct student s1, s2. The semicolon after the closing brace is mandatory. Members can be of any valid data type.

3.4.3 Memory allocation for structures

Theory

When a structure variable is declared, memory is allocated for **all members** sequentially (with possible padding for alignment). Total size may be **greater than sum of member sizes** due to alignment rules on the processor.

Example: struct with char (1 byte) + int (4 bytes) may occupy 8 bytes due to padding, not 5.

Use `sizeof(struct tag)` to get actual size. Structure members are stored contiguously in definition order (implementation-defined padding).

Array of structures: `struct student class[40];` — 40 student records contiguous in memory. Pointer to structure holds address of entire structure.

Important Points

- Memory allocated for **all members**
- Size may include **padding** for alignment
- `sizeof(struct tag)` gives actual bytes
- Size $\geq$ sum of member sizes
- Array of struct: contiguous records
- Padding depends on compiler/platform

For Exam

When a structure variable is declared, memory is allocated for all its members. The size may be larger than the sum of individual member sizes due to padding for memory alignment. `sizeof(struct name)` returns the actual size in bytes. An array of structures stores multiple records in contiguous memory.

3.4.4 Accessing structure members

Theory

Structure members are accessed using the **dot operator (.)**:

```
struct student s1;
s1.roll_no = 101;
strcpy(s1.name, "Ravi");
s1.marks = 85.5;
```

Left operand must be a structure variable; right operand is member name.

Cannot access entire structure at once for I/O in standard C — access members individually (except assignment between same-type structures: `s2 = s1;` copies all members).

Nested member access: `s1.dob.day` if dob is nested structure. Array of structures:

```
class[0].roll_no .
```

Important Points

- **Dot operator (.)** accesses members
- syntax: **variable.member**
- Left side = structure variable
- Cannot scanf/printf entire struct directly
- struct assignment copies all members
- Array: `arr[i].member`

For Exam

Structure members are accessed using the dot operator: `variable.member`. Example: `s1.roll_no = 101`. The left operand must be a structure variable. Entire structures can be copied with assignment (`s2 = s1`). For array of structures, use `arr[i].member` to access members.

3.4.5 Pointers to structures

Theory

A pointer can point to a structure:

```
struct student s1, *ptr;  
ptr = &s1;
```

Members accessed via pointer using **arrow operator (→)**:

```
ptr->roll_no = 101;  
strcpy(ptr->name, "Ravi");
```

Equivalent to dereference + dot: `(*ptr).roll_no = 101;`

Arrow is shorthand when pointer points to structure. Common when passing structures to functions by pointer (efficient — avoids copying entire structure). Dynamic structures allocated with malloc also use structure pointers.

Important Points

- Declare: `struct tag *ptr;`
- Assign: `ptr = &structure_var;`
- Access: **ptr→member** (arrow operator)

- Equivalent: `(*ptr).member`
- Efficient for function parameters
- Used with dynamic allocation

For Exam

A pointer to a structure is declared as `struct tag ptr`. Members are accessed using the arrow operator: `ptr→member`, which is shorthand for `(ptr).member`. Structure pointers are efficient for passing structures to functions without copying the entire structure.

3.4.6 Array of structures

Theory

An **array of structures** stores multiple records of the same structure type:

```
struct student {
    char name[20];
    int roll;
    float marks;
} stud[50]; // 50 students
```

Access individual record: `stud[i]`

Access member: `stud[i].marks`, `strcpy(stud[i].name, "Anita")`

Used for databases in memory — student lists, employee records, inventory items.

Loop through array to process all records:

```
for (int i = 0; i < n; i++)
    printf("%s%d\n", stud[i].name, stud[i].roll);
```

Pass to function with array name and count; changes to elements affect original array.

Important Points

- Syntax: `struct tag arr[size];`
- Each element = complete structure record
- Access: `arr[i].member`
- Used for lists of records
- Loop to process all records
- Pass array name + count to functions

For Exam

An array of structures stores multiple records of the same structure type. Declared as `struct tag array_name[size]`. Individual records accessed as `arr[i]` and members as `arr[i].member`. Used for maintaining lists of students, employees, or similar records in memory.

3.4.7 Nested structures

Theory

A **nested structure** contains another structure as a member:

```
struct date {
    int day, month, year;
};
struct student {
    char name[20];
    struct date dob;
    int roll;
};
```

Access nested member: `s1.dob.day = 15;` or `s1.dob.month = 6;`

Multiple levels of nesting possible. Nested structures organise hierarchical data — address containing city/state, employee containing department structure.

Initialization of nested structures may use nested braces: `{ "Ali", {15, 6, 2000}, 101 }`.

Important Points

- Structure as member of another structure
- Access: **outer.inner.member**
- Example: `student.dob.day`
- Organises hierarchical data
- Nested braces for initialization
- Multiple nesting levels allowed

For Exam

A nested structure contains another structure as its member. Access nested members using multiple dots: `outer.inner.member`. Example: `student.dob.day` where `dob` is a struct `date` member. Nested structures represent hierarchical data relationships.

3.4.8 Passing structures to functions

Theory

Structures can be passed to functions:

By value: Entire structure copied to function parameter. Changes do not affect original. Inefficient for large structures.

```
void display(struct student s) {  
    printf("%s", s.name);  
}
```

By pointer (reference): Pass address; receive pointer. Efficient; changes affect original.

```
void update(struct student *s) {  
    s->marks = 90;  
}  
// Call: update(&s1);
```

Return structures from functions (by value) is allowed but copying cost applies. Pointer approach preferred for large structures.

Important Points

- **Pass by value:** copy entire struct (slow for large)
- **Pass by pointer:** pass &struct; param struct *s
- Pointer method allows modification
- Call: func(s1) vs func(&s1)
- Return struct by value possible
- Prefer pointers for efficiency

For Exam

Structures can be passed to functions by value (entire structure copied, changes not reflected) or by pointer (pass address with &, receive as struct , *changes reflected*). *Pointer passing is more efficient for large structures. Example: void update(struct student s) { s->marks = 90; }.*

3.4.9 Introduction to union

Theory

A **union** is a user-defined type similar to structure but with a critical difference: **all members share the same memory location**. Only one member holds a valid value at any time.

Used when memory saving is important and only one field is needed at a time — e.g., variant record that holds either int or float or char, but never more than one simultaneously.

Keyword: `union`. Size of union = size of **largest member** (not sum of members).

Writing to one member then reading another without re-assigning gives implementation-dependent garbage — programmer must track which member is active.

Important Points

- `union` keyword — shared memory
- All members share **same address**
- Size = **largest member** size
- Only **one member** valid at a time
- Memory efficient for variant data
- Must track active member

For Exam

A union is a user-defined data type where all members share the same memory location. Only one member can hold a valid value at a time. The size of a union equals the size of its largest member. Unions save memory when only one of several fields is needed at any moment.

3.4.10 Defining and using union

Theory

Union definition syntax mirrors structure:

```
union data {
    int i;
    float f;
    char c;
};
union data d;
```

Usage:

```
d.i = 10;      // i is active
d.f = 3.14;    // overwrites i's memory; f is now active
printf("%f", d.f);
```

Members accessed with dot or arrow (if pointer): `d.i`, `ptr->f`.

Cannot initialize multiple members simultaneously in standard C. Initialize first member: `union data d = {10};`

Common in embedded systems, protocol parsing, and memory-constrained applications.

Important Points

- Syntax: `union tag { members };`
- Access: **variable.member**
- Assigning one member overwrites others
- Initialize first member only
- Track which member is currently valid
- Pointer: `union *p; p->member`

For Exam

A union is defined as `union tag { members };` and accessed with the dot operator. Assigning to one member overwrites the memory used by others. Only the most recently assigned member should be read. Example: `d.i = 10;` then `d.f = 3.14;` makes `f` the active member.

3.4.11 Structure vs union

Theory

Comparison of structure and union:

Feature	Structure	Union
Keyword	<code>struct</code>	<code>union</code>
Memory	Separate space per member	Shared space
Size	Sum of members (+ padding)	Largest member
Members active	All simultaneously	One at a time
Use case	Full records	Memory-saving variants
Access	All members valid	Only last written valid

Choose structure when all fields needed together (student record). Choose union when fields are mutually exclusive alternatives (integer OR float OR char value in same slot).

Important Points

- struct: all members have own memory
- union: members share memory
- struct size $\geq$ sum of members
- union size = max member size
- struct: all fields valid together
- union: one field valid at a time

For Exam

Structure allocates separate memory for each member; union shares one memory location among all members. Structure size is approximately the sum of member sizes; union size equals the largest member. In structure all members are valid simultaneously; in union only one member is valid at a time.

3.4.12 typedef

Theory

typedef creates an alias (alternative name) for an existing data type, simplifying declarations:

```
typedef int Integer;
typedef struct student Student;
typedef struct student {
    char name[20];
    int roll;
} Student;
```

Now use `Student s1;` instead of `struct student s1;`

typedef does not create new type — creates synonym. Useful for structures, unions, pointers:

```
typedef int *IntPtr;
IntPtr p;
```

Improves code readability and portability. Common in professional C code and APIs.

Important Points

- **typedef** creates type alias
- Syntax: typedef existing_type new_name;
- With struct: typedef struct {...} Name;
- Then use Name without struct keyword
- Does not create new type — synonym only
- Improves readability

For Exam

typedef creates an alias for a data type. Example: typedef struct student { ... } Student; allows declaration as Student s1 instead of struct student s1. typedef does not create a new type but provides a convenient alternative name, improving code readability.

3.4.13 Enumeration (enum)

Theory

enum defines a set of named integer constants:

```
enum days { MON, TUE, WED, THU, FRI, SAT, SUN };  
enum days today = WED;
```

By default, values start at 0 and increment: MON=0, TUE=1, etc. Can assign explicit values: `enum { RED=1, GREEN, BLUE };`

enum improves readability over #define or magic numbers — `if (today == WED)` clearer than `if (today == 2)`.

enum type is compatible with int in C. Size typically sizeof(int). Used for menu choices, states, flags, colours.

Important Points

- **enum** defines named integer constants
- Default values: 0, 1, 2, ... incrementing
- Can assign explicit values
- Improves code readability
- enum compatible with int
- Used for states, days, menu options

For Exam

enum defines a set of named integer constants. By default values start at 0 and increment. Example: enum days { MON, TUE, WED }; where MON=0, TUE=1, WED=2. enum makes code more readable than using raw numbers for states or menu choices.

3.4.14 Applications of structures and union

Theory

Structure applications:

- Student/employee database records
- Graphics: point (x,y), rectangle dimensions
- Date and time handling
- Linked list nodes (data + next pointer)
- Configuration settings grouped together
- File record formats

Union applications:

- Memory-efficient variant types (int OR float OR char)
- Interpreting same bytes as different types
- Embedded systems with limited RAM
- Network protocol fields with type-dependent meaning
- Bit-field alternatives (with care)

Combined use: struct containing union for variant field within fixed record. Structures and unions are building blocks for advanced data structures in C.

Important Points

- Structures: records, graphics points, dates, linked lists
- Unions: variant data, memory saving, byte reinterpretation
- struct + union: record with variant field
- Foundation for linked lists, trees
- Used in file handling, databases
- Essential for real-world C applications

For Exam

Structures are used for student records, employee data, geometric points, dates, and linked list nodes. Unions are used when memory must be saved and only one of several types is needed at a time, such as in embedded systems. Together they enable complex data organization in C programs.

Block 4: Storage Classes, Files and Preprocessors

Unit 1: Storage Classes

4.1.1 Storage classes in C

Theory

A **storage class** specifies not only storage location but also **scope** (where the variable is visible), **lifetime** (how long it exists), and **default initial value** of a variable. C provides four storage classes: **auto**, **register**, **static**, and **extern**.

auto (automatic): Default for local variables. Stored in stack memory. Created when block entered, destroyed when block exits. Uninitialized value is **garbage** (undefined).

register: Suggests storing variable in CPU register for faster access. Compiler may ignore. Cannot take address with **&**. Same scope/lifetime as **auto**.

static: For local variables — retains value between function calls; initialized once to zero. For global scope — limits visibility to current file (internal linkage).

extern: Declares global variable defined elsewhere. Extends visibility across files. No new memory allocated — refers to existing definition.

Understanding storage classes is essential for predicting variable behaviour across function calls and source files.

Important Points

- Four storage classes: **auto**, **register**, **static**, **extern**
- Define **scope**, **lifetime**, **default value**
- **auto** — default local; garbage if uninitialized
- **register** — fast access hint; no **&**
- **static** — retains value; init once to 0
- **extern** — global declared in another file

For Exam

Storage classes in C define scope, lifetime, and default value of variables. The four storage classes are **auto** (default for locals, destroyed when block ends), **register**

(suggested CPU register storage), static (retains value between calls, initialized once to zero), and extern (declares global variable defined elsewhere).

4.1.2 Comparison of storage classes

Theory

Detailed comparison table for all four storage classes:

Storage Class	Keyword	Storage	Scope	Lifetime	Default
Automatic	auto	Memory (stack)	Within block	Block duration	Garbage
Register	register	CPU register (if available)	Within block	Block duration	Garbage
Static (local)	static	Memory	Within block	Entire program	Zero
Static (global)	static	Memory	Current file	Entire program	Zero
External	extern	Memory	Entire program (across files)	Entire program	Zero

Key behaviours for exams:

- Local auto: recreated every call; value lost between calls
- Local static: keeps value; count++ in static counter example prints 1, 2, 3...
- Global variables: accessible from all functions (unless static limits file scope)
- extern in file2.c: `extern int x;` uses x defined in file1.c
- static cannot be used as function parameter storage class

Important Points

Class	Scope	Lifetime	Default
auto	Block	Block	Garbage
register	Block	Block	Garbage
static (local)	Block	Program	0
extern	Program	Program	0

- auto default inside functions
- static counter retains value between calls
- extern links globals across files
- Global without static visible to all files
- Local shadows global of same name

For Exam

The four storage classes differ in scope, lifetime, and default values. `auto` and `register` variables are local with block lifetime and garbage default. `static` local variables retain values between function calls and default to zero. `extern` declares globals defined in other files. `static` global limits visibility to the current file.

```
void counter() {
    static int count = 0;
    count++;
    printf("%d ", count);
}
/* Calls print: 1 2 3 */
```

Unit 2: Managing Files

4.2.1 Introduction to files

Theory

A **file** is a named collection of data stored permanently on secondary storage (hard disk, SSD). File handling allows programs to persist data beyond program execution — unlike variables that vanish when the program ends.

C supports two file types:

- **Text files:** Data stored as ASCII characters; human-readable (.txt, .c). Operations translate between text and internal binary representation.
- **Binary files:** Raw bytes stored exactly as in memory (.dat, .bin). Faster and more compact but not human-readable.

File operations in C: **create/open, read, write, close, seek** (position). The `<stdio.h>` library provides file functions using `FILE` structure and `FILE*` pointer.

Files enable databases, configuration storage, logging, report generation, and data exchange between programs.

Important Points

- File = permanent data storage on disk
- **Text file** — ASCII, human-readable
- **Binary file** — raw bytes, compact
- FILE pointer (FILE *fp) represents open file
- Operations: open, read, write, close, seek
- Data persists after program terminates

For Exam

Files store data permanently on disk. C supports text files (ASCII, human-readable) and binary files (raw binary data). File operations include open, read, write, close, and seek. The FILE pointer type represents an open file stream. File handling enables data persistence beyond program execution.

4.2.2 Opening and closing files

Theory

Before file operations, a file must be **opened** using `fopen()`. After operations complete, it must be **closed** using `fclose()`.

```
FILE *fp;
fp = fopen("filename.txt", "mode");
if (fp == NULL) {
    printf("Error opening file");
    exit(1);
}
/* file operations */
fclose(fp);
```

fopen(filename, mode): Returns pointer to FILE structure on success, **NULL** on failure (file not found, permission denied, path invalid). Always check for **NULL** before using.

fclose(fp): Closes file, flushes buffers (writes pending data), releases resources. Failing to close may lose data still in buffer. Cannot read/write after `fclose` on same pointer without reopening.

Declare `FILE *fp;` before `fopen`. Multiple files can be open simultaneously with different FILE pointers.

Important Points

- **fopen(name, mode)** — open file; returns `FILE*` or **NULL**
- **fclose(fp)** — close file; flush buffer
- Always **check NULL** after `fopen`
- `FILE *fp` declares file pointer
- `fclose` ensures data saved to disk
- Reopen required after `fclose` for reuse

For Exam

Files are opened with `fopen(filename, mode)` which returns a FILE pointer or NULL on failure. Always check for NULL before using the file. Files must be closed with `fclose(fp)` after operations to flush buffers and release resources. Forgetting `fclose` may cause data loss.

4.2.3 File open modes

Theory

The mode string in `fopen` determines how the file is opened:

Mode	Meaning
"r"	Read existing text file; fails if not exists
"w"	Write text file; creates new or truncates existing
"a"	Append to text file; creates if not exists
"r+"	Read and write existing text file
"w+"	Read/write; creates or truncates
"a+"	Read and append
"rb"	Read binary
"wb"	Write binary
"ab"	Append binary
"r+b" / "rb+"	Read/write binary

"w" warning: Opens file and erases previous content. Use "a" to add without destroying. Add **b** for binary mode on Windows (optional on Unix).

Reading from write-only or writing to read-only mode causes undefined behaviour or runtime error.

Important Points

- **"r"** read only; file must exist
- **"w"** write; creates/truncates (erases old data)
- **"a"** append; add to end
- **"r+"** read+write existing
- **"w+"** read+write; truncates
- **"a+"** read+append
- Add **b** for binary: "rb", "wb", "ab"
- Match mode to intended operation

For Exam

File open modes in `fopen` include: "r" (read existing text file), "w" (write, creates or truncates), "a" (append), "r+" (read and write existing), "w+" (read/write, truncates), and "a+" (read and append). Binary modes add b: "rb", "wb", "ab". Mode "w" erases existing file content.

4.2.4 Reading and writing — `fprintf` and `fscanf`

Theory

Formatted file I/O mirrors `printf/scanf` but uses a file pointer:

`fprintf(fp, format, args...)` — write formatted data to file.

`fscanf(fp, format, &args...)` — read formatted data from file.

```
fprintf(fp, "%d%s%f\n", roll, name, marks);
fscanf(fp, "%d%s%f", &roll, name, &marks);
```

Same format specifiers as `printf/scanf` (`%d`, `%f`, `%s`, etc.). `fscanf` returns EOF when no more matching input.

Also available:

- **`fgets(str, size, fp)`** — read line from file

- **`fputs(str, fp)`** — write string to file

Formatted I/O suitable for text files with structured data (records with fields). Binary files use `fread/fwrite` (beyond basic SLM scope but related).

Important Points

- **`fprintf(fp, format, args)`** — formatted write to file
- **`fscanf(fp, format, &args)`** — formatted read from file
- Same format specifiers as `printf/scanf`
- `fscanf` returns **EOF** when input exhausted
- `fgets/fputs` for line-based string I/O
- Suitable for text files with structured records

For Exam

`fprintf` writes formatted output to a file using a format string and file pointer, similar to `printf`. `fscanf` reads formatted input from a file, similar to `scanf`. Both use the same format specifiers. `fscanf` returns EOF when no more data matches. `fgets` and `fputs` handle line-based string I/O.

```
fp = fopen("data.txt", "w");
fprintf(fp, "Roll:%d, Marks:%.2f\n", 101, 85.5);
```

```
fclose(fp);

fp = fopen("data.txt", "r");
fscanf(fp, "Roll:%d, Marks:%f", &r, &m);
fclose(fp);
```

4.2.5 Character I/O — getc and putc

Theory

Character-level file I/O reads and writes one character at a time:

getc(fp) — reads one character from file; returns char as int or **EOF** (-1) at end of file.

putc(c, fp) — writes character c to file; returns written char or EOF on error.

```
int ch;
while ((ch = getc(fp)) != EOF)
    putchar(ch);    // display on screen
```

Or copy file character by character:

```
while ((ch = getc(fsource)) != EOF)
    putc(ch, ftarget);
```

Character I/O works for both text and binary files. **EOF (End Of File)** is defined as **-1** in **stdio.h** — signals no more data. Essential for file copy utilities and text processing.

Important Points

- **getc(fp)** — read one character; returns EOF at end
- **putc(c, fp)** — write one character
- **EOF = -1** (end of file marker)
- Loop until getc returns EOF
- Used for file copy, character processing
- Works on text and binary files

For Exam

getc reads one character from a file and returns it as an integer, or EOF (-1) at end of file. putc writes a single character to a file. A common pattern reads characters in a loop until EOF. EOF indicates end of file. These functions are used for character-by-character file processing and file copying.

```
int ch;
while ((ch = getc(fp)) != EOF)
    putc(ch, stdout);
```

4.2.6 File positioning — fseek, ftell and rewind

Theory

File operations read/write sequentially by default. **File positioning** functions move the internal file pointer:

fseek(fp, offset, position): Moves file pointer. position: **SEEK_SET (0)** start of file, **SEEK_CUR (1)** current position, **SEEK_END (2)** end of file. offset: bytes to move (can be negative).

Examples:

- `fseek(fp, 0, SEEK_SET)` — go to beginning
- `fseek(fp, 0, SEEK_END)` — go to end
- `fseek(fp, 10, SEEK_CUR)` — skip 10 bytes forward

ftell(fp): Returns current file pointer position as long integer (byte offset from start).

rewind(fp): Equivalent to `fseek(fp, 0, SEEK_SET)` — resets to beginning.

Used for random access in files, skipping headers, updating specific records, measuring file size.

Important Points

- **fseek(fp, offset, origin)** — move file pointer
- origin: **SEEK_SET(0)**, **SEEK_CUR(1)**, **SEEK_END(2)**
- **ftell(fp)** — current position in bytes
- **rewind(fp)** — go to start (= `fseek 0, SEEK_SET`)
- Enables random access in files
- Negative offset moves backward (with **SEEK_END/CUR**)

For Exam

fseek moves the file pointer to a specified position. The third argument is **SEEK_SET** (beginning), **SEEK_CUR** (current), or **SEEK_END** (end). ftell returns the current file position in bytes. rewind(fp) resets the file pointer to the beginning. These functions enable random access in files.

Unit 3: Command-line Arguments

4.3.1 argc, argv and atoi

Theory

Command-line arguments are values passed to a program when it is executed from the terminal/command prompt. Example: `./program 10 20` passes 10 and 20 as arguments.

C receives them through a special form of main:

```
int main(int argc, char *argv[])
```

argc (argument count): Integer — total number of arguments including program name. `./sum 4 7` → `argc = 3`.

argv (argument vector): Array of character pointers (strings). `argv[0]` = program name, `argv[1]` = first argument, `argv[2]` = second, etc. `argv[argc]` is NULL.

atoi(string): Converts ASCII string to integer (stdlib.h). `int n = atoi(argv[1]);`

Cannot do arithmetic on `argv[1]` directly — it is a string ("4" not 4). Use `atoi`, `atof` (float), or `sscanf` for conversion.

Useful for flexible programs: file copy (source dest names), calculator (operands from command line), configuration parameters.

Important Points

- **argc** — argument count (includes program name)
- **argv** — array of strings (`char *argv[]`)
- **argv[0]** = program name; **argv[1]** = first argument
- **argv[argc] = NULL**
- **atoi()** converts string argument to int
- Cannot use `argv[1]` as number without conversion

For Exam

Command-line arguments are passed to main through `argc` (argument count including program name) and `argv` (array of string pointers). `argv[0]` is the program name and `argv[1]`, `argv[2]` are user arguments. `atoi` from `stdlib.h` converts a string argument to integer. `argv` elements are strings and must be converted before numeric use.

```
int main(int argc, char *argv[]) {  
    if (argc == 3) {  
        int a = atoi(argv[1]);  
    }
```

```

    int b = atoi(argv[2]);
    printf("Sum =%d\n", a + b);
} else
    printf("Usage: program num1 num2\n");
return 0;
}
/* Run: ./sum 4 7 → Sum = 11 */

```

Unit 4: Macros and Preprocessor Directives

4.4.1 Macros

Theory

The **preprocessor** runs before compilation and processes lines starting with `#`. A **macro** is a name defined with `#define` that the preprocessor replaces with its definition (text substitution).

Object-like macro:

```

#define PI3.14159
#define MAX100

```

Every PI in code becomes 3.14159 before compilation.

Function-like macro:

```

#define SQUARE(x) ((x)*(x))
#define MAX(a, b) ((a)>(b)?(a):(b))

```

SQUARE(5) expands to ((5) * (5)).

Rules: No semicolon after `#define`. Use parentheses to avoid precedence bugs.

`#undef NAME` removes definition. Macros are not type-checked — faster but less safe than functions.

Predefined macros: `__DATE__`, `__TIME__`, `__FILE__`, `__LINE__`.

Important Points

- **#define NAME value** — object-like macro
- **#define FUNC(x) body** — function-like macro
- Preprocessor **text substitution** before compile
- No semicolon after `#define`

- Use parentheses in function-like macros
- **#undef** removes macro
- Macro vs function: macro faster, no type check

For Exam

Macros are defined using **#define** and replaced by the preprocessor before compilation. Object-like macros define constants: **#define PI 3.14159**. Function-like macros define code snippets: **#define SQUARE(x) ((x)*(x))**. Macros perform text substitution without type checking. They are faster than functions but can cause side-effect bugs.

```
#define PI3.14159
#define AREA(r) (PI* r* r)
/* AREA(5) expands to (3.14159 * 5 * 5) */
```

4.4.2 #include directive

Theory

The **#include** directive inserts the contents of another file into the source file during preprocessing. Two forms:

System header: `#include <stdio.h>` — searches system include directories for standard library headers.

User header: `#include "myheader.h"` — searches current project directory first, then system paths.

Header files (.h) typically contain: function prototypes, macro definitions, structure declarations, constant definitions. Including headers gives access to library functions without redefining them.

Common headers:

- `<stdio.h>` — I/O (printf, scanf, FILE, fopen)
- `<stdlib.h>` — memory (malloc, free), exit, atoi
- `<string.h>` — string functions
- `<math.h>` — mathematical functions

Multiple includes of same header guarded by include guards (`#ifndef` / `#define` / `#endif`) in professional code.

Important Points

- **#include <file.h>** — system header path
- **#include "file.h"** — user header (current dir first)

- Inserts header content at that point
- Required for library functions (stdio.h, etc.)
- No semicolon after #include
- stdio = Standard Input Output

For Exam

The #include directive inserts another file's contents during preprocessing. #include <stdio.h> includes system headers from standard directories. #include "file.h" includes user headers from the current directory first. Headers provide function prototypes and constants. stdio.h provides standard input-output functions.

4.4.3 Conditional compilation

Theory

Conditional compilation includes or excludes code blocks based on conditions evaluated at preprocess time:

```
#ifdef DEBUG
    printf("Debug: x =%d\n", x);
#endif

#ifndef BUFFER_SIZE
#define BUFFER_SIZE256
#endif

#if VERSION >= 2
    /* new code */
#else
    /* old code */
#endif
```

Directives:

- **#ifdef IDENT** — compile if macro defined
- **#ifndef IDENT** — compile if macro NOT defined
- **#if condition** — compile if condition true
- **#else** — alternative block
- **#endif** — end conditional block

Uses: debug code removal, platform-specific code, header include guards, feature toggling. Code in false branches is completely removed — not executed, not compiled.

Important Points

- **#ifdef / #ifndef** — compile if (not) defined
- **#if / #else / #endif** — conditional blocks
- Code in false branch **not compiled**
- Used for debug flags, platform differences
- Include guard: **#ifndef HEADER_H / #define HEADER_H**
- Preprocessor evaluates conditions, not compiler

For Exam

Conditional compilation uses directives like **#ifdef**, **#ifndef**, **#if**, **#else**, and **#endif** to include or exclude code at preprocess time. **#ifdef** includes code if a macro is defined. **#ifndef** includes code if a macro is not defined. Used for debug code, platform-specific compilation, and header include guards. Excluded code is not compiled at all.

```
#define DEBUG
#ifdef DEBUG
    printf("Debug mode on\n");
#endif

#ifndef MAX_SIZE
#define MAX_SIZE 100
#endif
```

Block 1 — Revision Summary

- ☐ Define problem-solving and list the four steps (analyse, design, code, test)
- ☐ Explain top-down vs bottom-up approaches with examples
- ☐ State characteristics of a valid algorithm (finiteness, definiteness, input, output, effectiveness)
- ☐ Draw flowchart symbols: oval, parallelogram, rectangle, diamond
- ☐ Compare machine language, assembly language, and high-level languages
- ☐ Differentiate compiler, interpreter, and assembler
- ☐ Name C's creator (Dennis Ritchie, 1972) and key features
- ☐ Describe structure of C program and compilation pipeline

- ☐ List six token types and 32 keywords rule
- ☐ State identifier rules and constant types
- ☐ Name basic data types and modifiers (short, long, signed, unsigned)
- ☐ Use all operator categories; explain integer division and modulus
- ☐ Predict ++a vs a++ results; write ternary expression
- ☐ Explain implicit vs explicit type conversion

Block 2 — Revision Summary

- ☐ Use printf and scanf with correct format specifiers and & in scanf
- ☐ List escape sequences (, and format specifiers (%d, %f, %c, %s)
- ☐ Explain sequential, selection, and repetition control
- ☐ Write if-else, else-if ladder, and switch with break
- ☐ Compare for, while, do-while (entry vs exit controlled)
- ☐ Explain break, continue, and why goto is avoided
- ☐ Declare 1D and 2D arrays; state index rules
- ☐ Explain strings as char arrays with '\0' terminator
- ☐ Use strlen, strcpy, strcmp, strcat
- ☐ Declare pointer, use & and *, explain pointer arithmetic
- ☐ Use malloc, calloc, realloc, free; explain memory leak

Block 3 — Revision Summary

- ☐ Explain modular programming and advantages of functions
- ☐ Differentiate built-in and user-defined functions
- ☐ Write function with declaration, definition, and call
- ☐ Explain nested function calls (function calling function)
- ☐ Write recursive factorial/Fibonacci with base case
- ☐ Compare direct and indirect recursion
- ☐ Explain passing arrays to functions
- ☐ Differentiate local and global variables; explain shadowing
- ☐ Compare call by value and call by reference with swap example

- ☐ Define structure, access with . and →
- ☐ Explain array of structures and nested structures
- ☐ Compare structure vs union (memory, size, usage)
- ☐ Use typedef and enum

Block 4 — Revision Summary

- ☐ List four storage classes: auto, register, static, extern
- ☐ Compare scope, lifetime, and default values in a table
- ☐ Explain static local variable retaining value between calls
- ☐ Differentiate text and binary files
- ☐ Open and close files with fopen/fclose; check NULL
- ☐ List file modes: r, w, a, r+, w+, a+ and binary variants
- ☐ Use fprintf, fscanf, getc, putc
- ☐ Explain fseek, ftell, rewind for file positioning
- ☐ Define argc and argv; explain argv[0] vs argv[1]
- ☐ Use atoi to convert command-line string to integer
- ☐ Define object-like and function-like macros
- ☐ Explain #include, #define, and conditional compilation (#ifdef)
- ☐ Compare macros vs functions (speed, type checking, size)

Quick Reference — Exam Essentials

Topic	Must-Know
Problem-solving	Analyse → Design → Code → Test
Algorithm	Finite, unambiguous; input, process, output
Flowchart	Oval=start/stop, rectangle=process, diamond=decision
Translators	Compiler (whole program), interpreter (line by line)
C basics	main(), semicolon, case-sensitive, #include
Data types	int, char, float, double, void
Operators	9/2=4; % remainder; ++a vs a++
scanf/printf	& required in scanf for variables
Control	if, switch (break!), for, while, do-while

Topic	Must-Know
Arrays	Index 0 to n-1; strings end with \0
Pointers	& address, * value; malloc/free
Functions	Prototype, definition, call
Recursion	Base case mandatory
Parameters	Call by value default; pointers for reference
struct vs union	All members vs shared memory
Storage class	auto, static, register, extern
Files	FILE *fp; fopen modes; fclose; EOF=-1
argc/argv	argc includes name; argv[0]=program
Preprocessor	#define, #include; runs before compiler

About the Author

Abdul Vahab A A

Website: <https://abdulvahabaa.in>

Thank you for using these study notes. I hope they make your Sem 1 C programming subject easier to understand and revise.

All the best for your exams and future in IT. Keep learning, keep coding, and never stop improving.

Duaon mein Yaad Rakhna.

If These Notes Helped You — Say Thanks on WhatsApp

If this file helped you study or pass your exams, a small thank-you message means a lot. You can copy and send this:

Hi Abdul, your BCA C programming study notes were really helpful. Thank you so much! Wishing you success too.

Share your feedback or suggestions anytime — it helps improve notes for other students too.

Study notes based on SGOU SLM — Problem Solving and Programming in C (B21CA02DC). Covers Blocks 1–4, 16 Units, theory only.

Prepared by Abdul Vahab A A | abdulvahabaa.in